

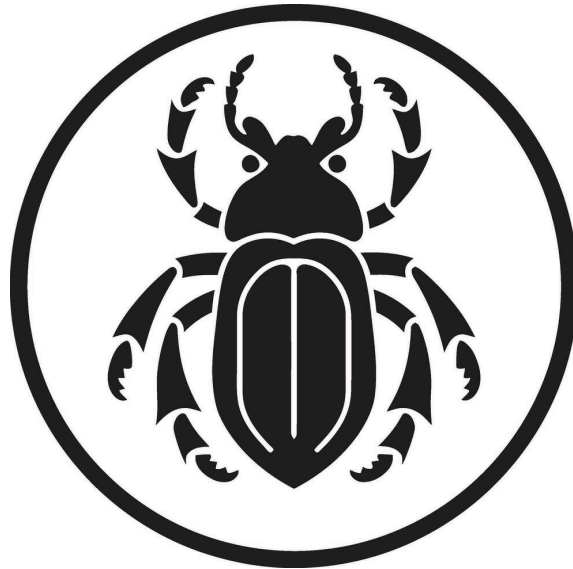


UNSA

Universidad Nacional de San Agustín

**“AÑO DE LA RECUPERACIÓN Y CONSOLIDACIÓN DE LA ECONOMÍA
PERUANA”**

**FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS**



Ejecución de Pruebas de Integración con Postman

ASIGNATURA:

Pruebas de Software

DOCENTE:

Ing. Robert Edison Arisaca Mamani

INTEGRANTES:

Anthony Leonel Chaisa Fernandez (Test Lead)

Champi Sanchez Manuel Mario (Test Analyst)

Huarca Pallani Jael Emerson (Test QA)

Chambi Orosco Luis Fernando (Test QA)

Chahua Charca Luis Ayben (Test QA)

Cahui Benegas Anthony Ronaldo (Test QA)

1. Resultados de ejecución	4
1.1. Sub-Módulo: Medication (endpoint /medication/)	4
1.1.1. TC01: Validar listado de medicación (GET)	4
1.1.2. TC02 - Validar creación de medicación (POST)	5
1.1.3. TC03 - Validar actualización de medicación (PATCH)	6
1.1.4. TC04 - Validar eliminación de medicación (DELETE)	8
1.2. Sub-Módulo: Note (endpoint /notes/)	9
1.2.1. TC05 - Validar listado de notas (GET)	9
1.2.2. TC06 - Validar creación de nota con hora explícita (POST)	10
1.2.3. TC07 - Validar creación de nota sin hora (POST)	11
1.2.4. TC08 - Validar actualización de nota (PATCH)	12
1.3. Sub-Módulo: Profile (endpoint /profile/)	13
1.3.1. TC09 - Validar perfil de usuario (GET)	13
1.4. Sub-Módulo: Sleep(endpoint /sleep/)	14
1.4.1. TC10 - Validar listado de registros de sueño (GET)	14
1.4.2. TC11 - Validar creación de registro de sueño con hora explícita (POST)	15
1.4.3. TC12 - Validar actualización de fin de sueño (PATCH)	16
1.5. Sub-Módulo: Tags (endpoint /tags/)	18
1.5.1. TC13 - Validar listado de tags (GET)	18
1.5.2. TC14 - Validar creación de tag (POST)	18
1.5.3. TC15 - Validar actualización de tag (PATCH)	19
1.5.4. TC16 - Validar eliminación de tag (DELETE)	20
1.5.5. TC17 - Validar creación de nota con tags (POST)	21
1.6. Sub-Módulo: Temperature (endpoint /temperature/)	22
1.6.1. TC18 - Validar listado de temperaturas (GET)	22
1.6.2. TC19 - Validar creación de temperatura con hora explícita (POST)	23
1.6.3. TC20 - Validar creación de temperatura sin hora (POST)	24
1.6.4. TC21 - Validar actualización de temperatura (PATCH)	25
1.7. Sub-Módulo: Timer (endpoint /timer/)	26
1.7.1. TC22 - Validar listado de timers (GET)	26
1.7.2. TC23 - Validar creación de timer (POST)	27
1.7.3. TC24 - Validar creación de timer sin usuario explícito (POST)	28
1.7.4. TC25 - Validar actualización de timer (PATCH)	29
1.7.5. TC26 - Validar reinicio de timer (PATCH)	30
1.8. Sub-Módulo: Tummitime (endpoint /Tummitime/)	31
1.8.1. TC27 - GET tummy times returns list	31
1.8.2. TC28 - Validar creación de tummy time con duración (POST)	32
1.8.3. TC29 - Validar actualización de milestone de tummy time (PATCH)	33
1.9. Sub-Módulo: Weight (endpoint /Weight/)	34
1.9.1. TC30 - Validar listado de peso (GET)	34
1.9.2. TC31 - Validar creación de peso con fecha explícita (POST)	35
1.9.3. TC32 - Validar creación de peso sin fecha (POST)	36
1.9.4. TC33 - Validar actualización de peso (PATCH)	37
1.10. Sub-Módulo: BMI (endpoint /bmi/)	39

1.10.1. TC34 — Validar listado de BMI (GET)	39
1.10.2. TC35 — Validar creación de registro de BMI (POST)	40
1.10.3. TC36 — Validar actualización parcial de BMI (PATCH)	41
1.10.4. TC37 — Validar eliminación de registro de BMI (DELETE)	42
1.11. Sub-Módulo: Child (endpoint /children/)	43
1.11.1. TC38 — Validar listado de Child (GET)	43
1.11.2. TC39 — Validar creación de registro de Child (POST)	44
1.11.3. TC40 — Validar actualización parcial de Child (PATCH)	46
1.11.4. TC41 — Validar eliminación de registro de Child (DELETE)	47
1.12. Sub-Módulo: Pumping (endpoint /pumping/)	48
1.12.1. TC42 — Validar listado de Pumping (GET)	48
1.12.2. TC43 — Validar creación de registro de Pumping (POST)	49
1.12.3. TC44 — Validar actualización parcial de Pumping (PATCH)	50
1.12.4. TC45 — Validar eliminación de registro de Pumping (DELETE)	52
1.13. Sub-Módulo: DiaperChange (endpoint /changes/)	53
1.13.1. TC46 — Validar listado de DiaperChange (GET)	53
1.13.2. TC47 — Validar creación de registro de DiaperChange (POST)	54
1.13.3. TC48 — Validar actualización parcial de DiaperChange (PATCH)	55
1.13.4. TC49 — Validar eliminación de registro de DiaperChange (DELETE)	56
1.14. Sub-Módulo: Feeding (endpoint /feedings/)	57
1.14.1. TC50 — Validar listado de Feeding (GET)	57
1.14.2. TC51 — Validar creación de registro de Feeding (POST)	58
1.14.3. TC52 — Validar actualización parcial de Feeding (PATCH)	59
1.14.4. TC53 — Validar eliminación de registro de Feeding (DELETE)	61
1.15. Sub-Módulo: HeadCircumference (endpoint /head-circumference/)	61
1.15.1. TC54 — Validar listado de HeadCircumference (GET)	61
1.15.2. TC55 — Validar creación de registro de HeadCircumference (POST)	63
1.15.3. TC56 — Validar actualización parcial de HeadCircumference (PATCH)	64
1.15.4. TC57 — Validar eliminación de registro de HeadCircumference (DELETE)	65
1.16. Sub-Módulo: Height (endpoint /height/)	66
1.16.1. TC58 — Validar listado de Height (GET)	66
1.16.2. TC59 — Validar creación de registro de Height (POST)	67
1.16.3. TC60 — Validar actualización parcial de Height (PATCH)	68
1.16.4. TC61 — Validar eliminación de registro de Height (DELETE)	69
1. Conclusiones	70
1.1. Total de casos de prueba ejecutados	70
1.2. Conclusiones	72

1. Resultados de ejecución

1.1. Sub-Módulo:Medication (endpoint /medication/)

1.1.1. TC01: Validar listado de medicación (GET)

Campo	Detalle
ID	TC01
Nombre	Validar listado de medicación
RF asociado	-
Descripción	Verifica que el endpoint devuelve la lista de registros de medicación.
Objetivo	Confirmar que GET /api/medication/ responde con HTTP 200 y JSON válido.
Precondiciones	Cuenta autenticada válida. sessionid y csrftoken disponibles.

Datos de entrada	Método: GET URL: https://ps-pruebas.onrender.com/api/medication/ Headers: Accept: application/json, Cookie, X-CSRFToken
Postcondiciones	La sesión autenticada permanece activa.
Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 200 OK con JSON que contiene results como arreglo.
Resultado Obtenido	HTTP 200 OK con lista de medicaciones.
Observaciones	No requiere body; solo autenticación válida.
Evidencia	
<pre>test('GET medications', async () => { const headers = await getHeaders(); const response = await apiFetch('/api/medication/', { method: 'GET', headers, }); expect(response.status).toBe(200); });</pre> Running 4 tests using 1 worker ✓ 1 [chromium] › tests\medication.spec.js:53:1 › GET medications (322ms)	

1.1.2. TC02 - Validar creación de medicación (POST)

Campo	Detalle
ID	TC02
Nombre	Validar creación de medicación
RF asociado	-
Descripción	Verifica que el endpoint crea un nuevo registro de medicación.
Objetivo	Confirmar que POST /api/medication/ responde con HTTP 201 y devuelve el recurso creado.
Precondiciones	Cuenta autenticada válida. child válido existente.
Datos de entrada	Método: POST URL: https://ps-pruebas.onrender.com/api/medication/ Headers: Content-Type: application/json, Accept: application/json, X-CSRFToken, Cookie Body JSON: child, name, dosage, dosage_unit, time, next_dose_interval, notes

Postcondiciones	Se crea un nuevo registro de medicación.
Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 201 Created con JSON que incluye id y los datos enviados.
Resultado Obtenido	HTTP 201 Created y registro retornado correctamente.
Observaciones	El campo time puede ser opcional según el comportamiento del API.
Evidencia	
<pre>test('POST medication', async () => { const childId = await createTestChild(); const headers = await getHeaders(); const response = await apiFetch('/api/medication/', { method: 'POST', headers, body: JSON.stringify({ child: childId, name: 'Tylenol', dosage: '5.0', dosage_unit: 'ml', time: '2017-11-18T12:00:00-05:00', next_dose_interval: '04:00:00', notes: 'For fever', }), }); expect(response.status).toBe(201); });</pre> ✓ 2 [chromium] > tests\medication.spec.js:62:1 > POST medication (405ms)	

1.1.3. TC03 - Validar actualización de medicación (PATCH9)

Campo	Detalle
ID	TC03
Nombre	Validar actualización de medicación
RF asociado	-
Descripción	Verifica que el endpoint actualiza parcialmente un registro de medicación.

Objetivo	Confirmar que PATCH /api/medication/{id}/ responde con HTTP 200 y persiste los cambios.
Precondiciones	Cuenta autenticada válida. Registro de medicación existente.
Datos de entrada	Método: PATCH URL: https://ps-pruebas.onrender.com/api/medication/{id} Headers: Content-Type: application/json, Accept: application/json, X-CSRFToken, Cookie Body JSON: dosage, notes
Postcondiciones	El registro de medicación se actualiza en el sistema.
Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 200 OK con JSON que refleja los cambios solicitados.
Resultado Obtenido	HTTP 200 OK y valores actualizados devueltos correctamente.
Observaciones	Solo se actualizan los campos enviados.

Evidencia

```
test('PATCH medication', async () => {
  const childId = await createTestChild();
  const headers = await getHeaders();
  const create = await apiFetch('/api/medication/', {
    method: 'POST',
    headers,
    body: JSON.stringify({
      child: childId,
      name: 'Medicine',
      dosage: '2.5',
      dosage_unit: 'ml',
    }),
  });
  expect(create.status).toBe(201);
  const created = await create.json();

  const response = await apiFetch(`/api/medication/${created.id}/`, {
    method: 'PATCH',
    headers,
    body: JSON.stringify({
      dosage: '5.0',
      notes: 'Updated',
    }),
  });
  expect(response.status).toBe(200);
});
```

```
✓ 3 [chromium] > tests\medication.spec.js:81:1 > PATCH medication (581ms)
```

1.1.4. TC04 - Validar eliminación de medicación (DELETE)

Campo	Detalle
ID	TC04
Nombre	Validar eliminación de medicación
RF asociado	-
Descripción	Verifica que el endpoint elimina un registro de medicación existente.
Objetivo	Confirmar que DELETE /api/medication/{id}/ responde con HTTP 204 y remueve el recurso.
Precondiciones	Cuenta autenticada válida. Registro de medicación existente.
Datos de entrada	Método: DELETE URL: https://ps-pruebas.onrender.com/api/medication/{id}/ Headers: Accept: application/json, X-CSRFToken, Cookie
Postcondiciones	El registro de medicación ya no debe existir.
Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 204 No Content sin body.
Resultado Obtenido	HTTP 204 No Content.
Observaciones	Verificar eliminación mediante código 204.
Evidencia	


```

test('DELETE medication', async () => {
  const childId = await createTestChild();
  const headers = await getHeaders();
  const create = await apiFetch('/api/medication/', {
    method: 'POST',
    headers,
    body: JSON.stringify({
      child: childId,
      name: 'Medicine',
    }),
  });
  expect(create.status).toBe(201);
  const created = await create.json();

  const response = await apiFetch(`/api/medication/${created.id}/`, {
    method: 'DELETE',
    headers,
  });
  expect(response.status).toBe(204);
});

```

✓ 4 [chromium] › tests\medication.spec.js:108:1 › DELETE medication (570ms)

1.2. Sub-Módulo:Note (endpoint /notes/)

1.2.1. TC05 - Validar listado de notas (GET)

Campo	Detalle
ID	TC05
Nombre	Validar listado de notas
RF asociado	-
Descripción	Verifica que el endpoint devuelve la lista de notas existentes.
Objetivo	Confirmar que GET /api/notes/ responde con HTTP 200 y JSON válido.
Precondiciones	Cuenta autenticada válida. sessionid y csrftoken disponibles.
Datos de entrada	Método: GET URL: https://ps-pruebas.onrender.com/api/notes/ Headers: Accept: application/json, Cookie, X-CSRFToken
Postcondiciones	La sesión autenticada permanece activa.
Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 200 OK con JSON que contiene results como arreglo.
Resultado Obtenido	HTTP 200 OK con lista de notas.
Observaciones	No requiere body; solo autenticación válida.

Evidencia
<pre>test('GET notes returns list', async ({ page }) => { const csrfToken = await getCsrfToken(page); const response = await page.request.get('/api/notes/', { headers: jsonHeaders(csrfToken), }); expect(response.status()).toBe(200); const body = await response.json(); expect(Array.isArray(body.results)).toBe(true); });</pre> Running 4 tests using 1 worker ✓ 1 [chromium] › tests\note.spec.js:5:3 › Note API › GET notes returns list (1.6s)

1.2.2. TC06 - Validar creación de nota con hora explícita (POST)

Campo	Detalle
ID	TC06
Nombre	Validar creación de nota con hora explícita
RF asociado	-
Descripción	Verifica que el endpoint crea una nota cuando se proporciona el campo time.
Objetivo	Confirmar que POST /api/notes/ responde con HTTP 201 y devuelve la nota creada.
Precondiciones	Cuenta autenticada válida. child válido existente.
Datos de entrada	Método: POST URL: https://ps-pruebas.onrender.com/api/notes/ Headers: Content-Type: application/json, Accept: application/json, X-CSRFToken, Cookie Body JSON: child, note, time
Postcondiciones	Se crea un nuevo registro de nota.
Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 201 Created con JSON que incluye note y time.
Resultado Obtenido	HTTP 201 Created y nota retornada correctamente.
Observaciones	El campo time es explícito en este caso.
Evidencia	

```

test('POST note with explicit time', async ({ page }) => {
  const childId = await createChild(page);
  const csrfToken = await getCsrftoken(page);
  const payload = {
    child: childId,
    note: 'New fake note from Playwright',
    time: '2017-11-18T22:45:00-05:00',
  };
  const response = await page.request.post('/api/notes/', {
    headers: jsonHeaders(csrfToken),
    data: payload,
  });
  expect(response.status()).toBe(201);
  const body = await response.json();
  expect(body.note).toBe(payload.note);
});

```

✓ 2 [chromium] > tests\note.spec.js:15:3 > Note API > POST note with explicit time (2.7s)

1.2.3. TC07 - Validar creación de nota sin hora (POST)

Campo	Detalle
ID	TC07
Nombre	Validar creación de nota sin hora
RF asociado	-
Descripción	Verifica que el endpoint crea una nota sin proporcionar el campo time.
Objetivo	Confirmar que POST /api/notes/ responde con HTTP 201 y devuelve la nota creada.
Precondiciones	Cuenta autenticada válida. child válido existente.
Datos de entrada	Método: POST URL: https://ps-pruebas.onrender.com/api/notes/ Headers: Content-Type: application/json, Accept: application/json, X-CSRFToken, Cookie Body JSON: child, note
Postcondiciones	Se crea un nuevo registro de nota.
Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 201 Created con JSON que incluye note.

Resultado Obtenido	HTTP 201 Created y nota retornada correctamente.
Observaciones	time no es obligatorio para la creación de nota.
Evidencia	
<pre>test('POST note without time', async ({ page }) => { const childId = await createChild(page); const csrfToken = await getCsrftoken(page); const payload = { child: childId, note: 'Another fake note from Playwright', }; const response = await page.request.post('/api/notes/', { headers: jsonHeaders(csrfToken), data: payload, }); expect(response.status()).toBe(201); const body = await response.json(); expect(body.note).toBe(payload.note); });</pre> ✓ 3 [chromium] > tests\note.spec.js:32:3 > Note API > POST note without time (2.0s)	

1.2.4. TC08 - Validar actualización de nota (PATCH)

Campo	Detalle
ID	TC08
Nombre	Validar actualización de nota
RF asociado	-
Descripción	Verifica que el endpoint actualiza el texto de una nota existente.
Objetivo	Confirmar que PATCH /api/notes/{id}/ responde con HTTP 200 y refleja el nuevo texto.
Precondiciones	Cuenta autenticada válida. Nota existente creada previamente.
Datos de entrada	Método: PATCH URL: https://ps-pruebas.onrender.com/api/notes/{id}/ Headers: Content-Type: application/json, Accept: application/json, X-CSRFToken, Cookie Body JSON: note
Postcondiciones	El texto de la nota se actualiza en el sistema.

Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 200 OK con JSON que muestra la nota actualizada.
Resultado Obtenido	HTTP 200 OK y texto actualizado correctamente.
Observaciones	Solo se modifica el campo note.
Evidencia	
<pre> test('PATCH note updates text', async ({ page }) => { const childId = await createChild(page); const csrfToken = await getCsrftoken(page); const createResponse = await page.request.post('/api/notes/', { headers: jsonHeaders(csrfToken), data: { child: childId, note: 'Note to update', time: '2017-11-18T22:45:00-05:00', }, }); expect(createResponse.status()).toBe(201); const created = await createResponse.json(); const patchResponse = await page.request.patch(`/api/notes/\${created.id}/`, { headers: jsonHeaders(csrfToken), data: { note: 'Updated note text.', }, }); expect(patchResponse.status()).toBe(200); const patched = await patchResponse.json(); expect(patched.note).toBe('Updated note text.');</pre> ✓ 4 [chromium] > tests\note.spec.js:48:3 > Note API > PATCH note updates text (2.1s)	

1.3. Sub-Módulo: Profile (endpoint /profile/)

1.3.1. TC09 - Validar perfil de usuario (GET)

Campo	Detalle
ID	TC09
Nombre	Validar perfil de usuario
RF asociado	-
Descripción	Verifica que el endpoint devuelve los datos de perfil del usuario autenticado.
Objetivo	Confirmar que GET /api/profile responde con HTTP 200 y devuelve user, language, timezone y api_key.

Precondiciones	Cuenta autenticada válida. sessionid y csrftoken presentes.
Datos de entrada	Método: GET URL: https://ps-pruebas.onrender.com/api/profile Headers: Accept: application/json, Cookie, X-CSRFToken
Postcondiciones	La sesión autenticada permanece activa.
Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 200 OK con JSON que incluye user.username, language, timezone y api_key.
Resultado Obtenido	HTTP 200 OK, perfil devuelto correctamente.
Observaciones	Usar GET /api/profile sin barra final; /api/profile/ falla en este servidor.
Evidencia	
<pre>test('GET profile returns user and api_key', async ({ page }) => { await getCsrftoken(page); const response = await page.request.get('/api/profile'); expect(response.status()).toBe(200); const body = await response.json(); expect(body.language).toBe('en-US'); expect(body.timezone).toBe('UTC'); expect(body.user.username).toBe('admin'); expect(typeof body.api_key).toBe('string'); expect(body.api_key.length).toBeGreaterThan(30); });</pre> Running 1 test using 1 worker ✓ 1 [chromium] › tests\profile.spec.js:5:3 › Profile API › GET profile returns user and api_key (1.3s)	

1.4. Sub-Módulo: Sleep(endpoint /sleep/)

1.4.1. TC10 - Validar listado de registros de sueño (GET)

Campo	Detalle
ID	TC10
Nombre	Validar listado de registros de sueño
RF asociado	-
Descripción	Verifica que el endpoint devuelve la lista de registros de sueño existentes.
Objetivo	Confirmar que GET /api/sleep/ responde con HTTP 200 y JSON válido.
Precondiciones	Cuenta autenticada válida. sessionid y csrftoken disponibles.

Datos de entrada	Método: GET URL: https://ps-pruebas.onrender.com/api/sleep/ Headers: Accept: application/json, Cookie, X-CSRFToken
Postcondiciones	La sesión autenticada permanece activa.
Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 200 OK con JSON que contiene results como arreglo de registros de sueño.
Resultado Obtenido	HTTP 200 OK con lista de registros de sueño.
Observaciones	No requiere body; solo autenticación válida.
Evidencia	
<pre>test('GET sleep returns results', async ({ page }) => { const csrfToken = await getCsrfToken(page); const response = await page.request.get('/api/sleep/', { headers: jsonHeaders(csrfToken), }); expect(response.status()).toBe(200); const body = await response.json(); expect(Array.isArray(body.results)).toBe(true); });</pre> Running 3 tests using 1 worker ✓ 1 [chromium] › tests\sleep.spec.js:5:3 › Sleep API › GET sleep returns results (1.5s)	

1.4.2. TC11 - Validar creación de registro de sueño con hora explícita (POST)

Campo	Detalle
ID	TC11
Nombre	Validar creación de registro de sueño con hora explícita
RF asociado	-
Descripción	Verifica que el endpoint crea un registro de sueño cuando se envía start, end y notes.
Objetivo	Confirmar que POST /api/sleep/ responde con HTTP 201 y calcula la duración correctamente.
Precondiciones	Cuenta autenticada válida. child válido existente.
Datos de entrada	Método: POST URL: https://ps-pruebas.onrender.com/api/sleep/ Headers: Content-Type: application/json, Accept: application/json,

	X-CSRFToken, Cookie Body JSON: child, start, end, notes
Postcondiciones	Se crea un nuevo registro de sueño con duración calculada.
Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 201 Created con JSON que incluye id, duration calculada y datos enviados.
Resultado Obtenido	HTTP 201 Created y registro con duración correcta.
Observaciones	La duración debe reflejar la diferencia entre start y end.
Evidencia	
<pre>test('POST sleep entry calculates duration', async ({ page }) => { const childId = await createChild(page); const csrfToken = await getCsrftoken(page); const response = await page.request.post('/api/sleep/', { headers: jsonHeaders(csrfToken), data: { child: childId, start: '2017-11-21T19:30:00-05:00', end: '2017-11-21T23:00:00-05:00', notes: 'used new swaddle', }, }); expect(response.status()).toBe(201); const body = await response.json(); expect(body.id).toBeTruthy(); expect(body.duration).toMatch(/3:30:00 03:30:00/); });</pre> ✓ 2 [chromium] › tests\sleep.spec.js:15:3 › Sleep API › POST sleep entry calculates duration (1.7s)	

1.4.3. TC12 - Validar actualización de fin de sueño (PATCH)

Campo	Detalle
ID	TC12
Nombre	Validar actualización de fin de sueño
RF asociado	-
Descripción	Verifica que el endpoint actualiza el campo end de un registro de sueño existente.
Objetivo	Confirmar que PATCH /api/sleep/{id}/ responde con HTTP 200 y devuelve el valor actualizado.

Precondiciones	Cuenta autenticada válida. Registro de sueño existente.
Datos de entrada	Método: PATCH URL: https://ps-pruebas.onrender.com/api/sleep/{id}/ Headers: Content-Type: application/json, Accept: application/json, X-CSRFToken, Cookie Body JSON: end
Postcondiciones	El registro de sueño se actualiza con el nuevo tiempo de finalización.
Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 200 OK con JSON que muestra el campo end actualizado.
Resultado Obtenido	HTTP 200 OK y end actualizado correctamente.
Observaciones	Se actualiza solo el campo end; la API puede devolver la hora en formato UTC.

Evidencia

```
test('PATCH sleep entry updates end time', async ({ page }) => {
  const childId = await createChild(page);
  const csrfToken = await getCsrfToken(page);
  const createResponse = await page.request.post('/api/sleep/', {
    headers: jsonHeaders(csrfToken),
    data: {
      child: childId,
      start: '2017-11-19T03:00:00-05:00',
      end: '2017-11-19T04:30:00-05:00',
      notes: 'testing patch',
    },
  },
  });
  expect(createResponse.status()).toBe(201);
  const created = await createResponse.json();

  const patchResponse = await page.request.patch(`/api/sleep/${created.id}/`, {
    headers: jsonHeaders(csrfToken),
    data: {
      end: '2017-11-19T08:30:00-05:00',
    },
  },
  );
  expect(patchResponse.status()).toBe(200);
  const patched = await patchResponse.json();
  expect(new Date(patched.end).toISOString()).toBe(
    new Date('2017-11-19T08:30:00-05:00').toISOString()
  );
});
```

✓ 3 [chromium] > tests\sleep.spec.js:33:3 > Sleep API > PATCH sleep entry updates end time (1.8s)

1.5. Sub-Módulo: Tags (endpoint /tags/)

1.5.1. TC13 - Validar listado de tags (GET)

Campo	Detalle
ID	TC13
Nombre	Validar listado de tags
RF asociado	-
Descripción	Verifica que el endpoint devuelve la lista de tags existentes.
Objetivo	Confirmar que GET /api/tags/ responde con HTTP 200 y JSON válido.
Precondiciones	Cuenta autenticada válida. sessionid y csrftoken disponibles.
Datos de entrada	Método: GET URL: https://ps-pruebas.onrender.com/api/tags/ Headers: Accept: application/json, Cookie, X-CSRFToken
Postcondiciones	La sesión autenticada permanece activa.
Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 200 OK con JSON que contiene results como arreglo de tags.
Resultado Obtenido	HTTP 200 OK con lista de tags.
Observaciones	No requiere body; solo autenticación válida.
Evidencia	
<pre>test('GET tags returns list', async ({ page }) => { const csrfToken = await getCsrfToken(page); const response = await page.request.get('/api/tags/', { headers: jsonHeaders(csrfToken), }); expect(response.status()).toBe(200); const body = await response.json(); expect(Array.isArray(body.results)).toBe(true); });</pre> Running 5 tests using 1 worker ✓ 1 [chromium] > tests\tags.spec.js:5:3 > Tag API > GET tags returns list (1.3s)	

1.5.2. TC14 - Validar creación de tag (POST)

Campo	Detalle
-------	---------

ID	TC14
Nombre	Validar creación de tag
RF asociado	-
Descripción	Verifica que el endpoint crea un nuevo tag con un nombre y color válidos.
Objetivo	Confirmar que POST /api/tags/ responde con HTTP 201 y devuelve el tag creado.
Precondiciones	Cuenta autenticada válida.
Datos de entrada	Método: POST URL: https://ps-pruebas.onrender.com/api/tags/ Headers: Content-Type: application/json, Accept: application/json, X-CSRFToken, Cookie Body JSON: name, color
Postcondiciones	Se crea un nuevo registro de tag.
Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 201 Created con JSON que incluye name y slug.
Resultado Obtenido	HTTP 201 Created y tag retornado correctamente.
Observaciones	El slug debe generarse automáticamente.
Evidencia	
<pre>test('POST tag creates new tag', async ({ page }) => { const csrfToken = await getCsrfToken(page); const response = await page.request.post('/api/tags/', { headers: jsonHeaders(csrfToken), data: { name: `new tag \${Date.now()}`, color: '■ #123456', }, }, {}); expect(response.status()).toBe(201); const body = await response.json(); expect(body.name).toContain('new tag'); expect(body.slug).toBeTruthy(); });</pre> ✓ 2 [chromium] › tests\tags.spec.js:15:3 › Tag API › POST tag creates new tag (1.0s)	

1.5.3. TC15 - Validar actualización de tag (PATCH)

Campo	Detalle
ID	TC15
Nombre	Validar actualización de tag
RF asociado	-
Descripción	Verifica que el endpoint actualiza un tag existente.
Objetivo	Confirmar que PATCH /api/tags/{slug}/ responde con HTTP 200 y guarda los cambios.
Precondiciones	Cuenta autenticada válida. Tag existente.
Datos de entrada	Método: PATCH URL: https://ps-pruebas.onrender.com/api/tags/{slug}/ Headers: Content-Type: application/json, Accept: application/json, X-CSRFToken, Cookie Body JSON: name, color
Postcondiciones	El tag se actualiza en el sistema.
Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 200 OK con JSON que refleja name y color actualizados.
Resultado Obtenido	HTTP 200 OK y tag actualizado correctamente.
Evidencia	
<pre>test('PATCH tag updates tag', async ({ page }) => { const slug = await createTag(page); const csrfToken = await getCsrftoken(page); const response = await page.request.patch(`/api/tags/\${slug}/`, { headers: jsonHeaders(csrfToken), data: { name: 'A different name', color: '■ #567890', }, }); expect(response.status()).toBe(200); const body = await response.json(); expect(body.name).toBe('A different name'); expect(body.color).toBe('■ #567890'); });</pre> ✓ 3 [chromium] > tests\tags.spec.js:30:3 > Tag API > PATCH tag updates tag (1.8s)	

1.5.4. TC16 - Validar eliminación de tag (DELETE)

Campo	Detalle
ID	TC16

Nombre	Validar eliminación de tag
RF asociado	-
Descripción	Verifica que el endpoint elimina un tag existente.
Objetivo	Confirmar que DELETE /api/tags/{slug}/ responde con HTTP 204 y remueve el recurso.
Precondiciones	Cuenta autenticada válida. Tag existente.
Datos de entrada	Método: DELETE URL: https://ps-pruebas.onrender.com/api/tags/{slug}/ Headers: Accept: application/json, X-CSRFToken, Cookie
Postcondiciones	El tag ya no debe existir.
Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 204 No Content sin body.
Resultado Obtenido	HTTP 204 No Content.
Evidencia	
<pre>test('DELETE tag removes tag', async ({ page }) => { const slug = await createTag(page); const csrfToken = await getCsrfToken(page); const response = await page.request.delete(`/api/tags/\${slug}/`, { headers: jsonHeaders(csrfToken), }); expect(response.status()).toBe(204); });</pre> ✓ 4 [chromium] › tests\tags.spec.js:46:3 › Tag API › DELETE tag removes tag (2.1s)	

1.5.5. TC17 - Validar creación de nota con tags (POST)

Campo	Detalle
ID	TC17
Nombre	Validar creación de nota con tags
RF asociado	-
Descripción	Verifica que al crear una nota con tags, el endpoint asigna correctamente los tags al recurso.
Objetivo	Confirmar que POST /api/notes/ responde con HTTP 201 y devuelve la nota con los tags asociados.
Precondiciones	Cuenta autenticada válida. child válido existente.
Datos de entrada	Método: POST URL: https://ps-pruebas.onrender.com/api/notes/ Headers: Content-Type: application/json, Accept: application/json, X-CSRFToken, Cookie Body JSON: child, note, tags

Postcondiciones	Se crea una nota con tags asociados.
Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 201 Created con JSON que incluye tags como arreglo.
Resultado Obtenido	HTTP 201 Created y tags asociados correctamente.
Evidencia	
<pre>test('POST tags to note creates tagged note', async ({ page }) => { const childId = await createChild(page); const csrfToken = await getCsrfToken(page); const payload = { child: childId, note: 'New tagged note.', tags: ['tag1', 'tag2'], }; const response = await page.request.post('/api/notes/', { headers: jsonHeaders(csrfToken), data: payload, }); expect(response.status()).toBe(201); const body = await response.json(); expect(body.tags).toEqual(expect.arrayContaining(['tag1', 'tag2'])); });</pre> ✓ 5 [chromium] › tests\tags.spec.js:55:3 › Tag API › POST tags to note creates tagged note (2.8s)	

1.6. Sub-Módulo: Temperature (endpoint /temperature/)

1.6.1. TC18 - Validar listado de temperaturas (GET)

Campo	Detalle
ID	TC18
Nombre	Validar listado de temperaturas
RF asociado	-
Descripción	Verifica que el endpoint devuelve la lista de registros de temperatura.
Objetivo	Confirmar que GET /api/temperature/ responde con HTTP 200 y JSON válido.
Precondiciones	Cuenta autenticada válida. sessionid y csrftoken disponibles.
Datos de entrada	Método: GET URL: https://ps-pruebas.onrender.com/api/temperature/ Header s: Accept: application/json, Cookie, X-CSRFToken
Postcondiciones	La sesión autenticada permanece activa.

Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 200 OK con JSON que contiene results como arreglo.
Resultado Obtenido	HTTP 200 OK con lista de temperaturas.
Evidencia	
<pre>test('GET temperatures returns list', async ({ page }) => { const csrfToken = await getCsrfToken(page); const response = await page.request.get('/api/temperature/', { headers: jsonHeaders(csrfToken), }); expect(response.status()).toBe(200); const body = await response.json(); expect(Array.isArray(body.results)).toBe(true); });</pre> Running 4 tests using 1 worker ✓ 1 [chromium] > tests\temperature.spec.js:5:3 > Temperature API > GET temperatures returns list (1.1s)	

1.6.2. TC19 - Validar creación de temperatura con hora explícita (POST)

Campo	Detalle
ID	TC19
Nombre	Validar creación de temperatura con hora explícita
RF asociado	-
Descripción	Verifica que el endpoint crea un registro de temperatura cuando se envía time.
Objetivo	Confirmar que POST /api/temperature/ responde con HTTP 201 y devuelve el recurso creado.
Precondiciones	Cuenta autenticada válida. child válido existente.
Datos de entrada	Método: POST URL: https://ps-pruebas.onrender.com/api/temperature/ Header s: Content-Type: application/json, Accept: application/json, X-CSRFToken, Cookie Body JSON: child, temperature, time, notes
Postcondiciones	Se crea un nuevo registro de temperatura.
Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 201 Created con JSON que incluye temperature y notes.

Resultado Obtenido	HTTP 201 Created y registro retornado correctamente.
Observaciones	El campo temperature se envía como número.
Evidencia	
<pre>test('POST temperature with explicit time', async ({ page }) => { const childId = await createChild(page); const csrfToken = await getCsrfToken(page); const data = { child: childId, temperature: 100.1, time: '2017-11-20T22:52:00-05:00', notes: 'rectal', }; const response = await page.request.post('/api/temperature/', { headers: jsonHeaders(csrfToken), data, }); expect(response.status()).toBe(201); const body = await response.json(); expect(body.temperature).toBe(data.temperature); expect(body.notes).toBe(data.notes); });</pre> ✓ 2 [chromium] > tests\temperature.spec.js:15:3 > Temperature API > POST temperature with explicit time (1.8s)	

1.6.3. TC20 - Validar creación de temperatura sin hora (POST)

Campo	Detalle
ID	TC20
Nombre	Validar creación de temperatura sin hora
RF asociado	-
Descripción	Verifica que el endpoint crea un registro de temperatura sin el campo time.
Objetivo	Confirmar que POST /api/temperature/ responde con HTTP 201 y devuelve el recurso creado.
Precondiciones	Cuenta autenticada válida. child válido existente.
Datos de entrada	Método: POST URL: https://ps-pruebas.onrender.com/api/temperature/ Header s: Content-Type: application/json, Accept: application/json, X-CSRFToken, Cookie Body JSON: child, temperature, notes
Postcondiciones	Se crea un nuevo registro de temperatura.

Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 201 Created con JSON que incluye temperature y notes.
Resultado Obtenido	HTTP 201 Created y registro retornado correctamente.
Observaciones	time no es obligatorio.
Evidencia	
<pre>test('POST temperature without time', async ({ page }) => { const childId = await createChild(page); const csrfToken = await getCsrfToken(page); const data = { child: childId, temperature: 100.5, notes: 'temporal', }; const response = await page.request.post('/api/temperature/', { headers: jsonHeaders(csrfToken), data, }); expect(response.status()).toBe(201); const body = await response.json(); expect(body.temperature).toBe(data.temperature); expect(body.notes).toBe(data.notes); });</pre> ✓ 3 [chromium] > tests\temperature.spec.js:34:3 > Temperature API > POST temperature without time (1.7s)	

1.6.4. TC21 - Validar actualización de temperatura (PATCH)

Campo	Detalle
ID	TC21
Nombre	Validar actualización de temperatura
RF asociado	-
Descripción	Verifica que el endpoint actualiza el valor de temperatura de un registro existente.
Objetivo	Confirmar que PATCH /api/temperature/{id}/ responde con HTTP 200 y devuelve el nuevo valor.
Precondiciones	Cuenta autenticada válida. Registro de temperatura existente.
Datos de entrada	Método: PATCH URL: https://ps-pruebas.onrender.com/api/temperature/{id}/ Headers: Content-Type: application/json, Accept:

	application/json, X-CSRFToken, Cookie Body JSON: temperature
Postcondiciones	El registro de temperatura se actualiza en el sistema.
Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 200 OK con JSON que muestra temperature actualizado.
Resultado Obtenido	HTTP 200 OK y valor actualizado correctamente.
Observaciones	temperature puede enviarse como número y ser devuelto como número.
Evidencia	
<pre> test('PATCH temperature updates value', async ({ page }) => { const childId = await createChild(page); const csrfToken = await getCsrftoken(page); const createResponse = await page.request.post('/api/temperature/', { headers: jsonHeaders(csrfToken), data: { child: childId, temperature: '98.6', time: '2017-11-17T12:52:00-05:00', notes: 'tympanic', }, }, {}); expect(createResponse.status()).toBe(201); const created = await createResponse.json(); const patchResponse = await page.request.patch(`/api/temperature/\${created.id}/`, { headers: jsonHeaders(csrfToken), data: { temperature: 99, }, }, {}); expect(patchResponse.status()).toBe(200); const patched = await patchResponse.json(); expect(patched.temperature).toBe(99); }); </pre> ✓ 4 [chromium] > tests\temperature.spec.js:52:3 > Temperature API > PATCH temperature updates value (1.9s)	

1.7. Sub-Módulo: Timer (endpoint /timer/)

1.7.1. TC22 - Validar listado de timers (GET)

Campo	Detalle
ID	TC22
Nombre	Validar listado de timers
RF asociado	-

Descripción	Verifica que el endpoint devuelve la lista de timers existentes.
Objetivo	Confirmar que GET /api/timers/ responde con HTTP 200 y JSON válido.
Precondiciones	Cuenta autenticada válida. sessionid y csrftoken disponibles.
Datos de entrada	Método: GET URL: https://ps-pruebas.onrender.com/api/timers/ Headers: Accept: application/json, Cookie, X-CSRFToken
Postcondiciones	La sesión autenticada permanece activa.
Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 200 OK con JSON que contiene results como arreglo de timers.
Resultado Obtenido	HTTP 200 OK con lista de timers.
Observaciones	No requiere body; solo autenticación válida.
Evidencia	
<pre>test('GET timers returns list', async ({ page }) => { const csrfToken = await getCsrfToken(page); const response = await page.request.get('/api/timers/', { headers: jsonHeaders(csrfToken), }); expect(response.status()).toBe(200); const body = await response.json(); expect(Array.isArray(body.results)).toBe(true); });</pre> Running 5 tests using 1 worker ✓ 1 [chromium] > tests\timer.spec.js:5:3 > Timer API > GET timers returns list (1.5s)	

1.7.2. TC23 - Validar creación de timer (POST)

Campo	Detalle
ID	TC23
Nombre	Validar creación de timer
RF asociado	-
Descripción	Verifica que el endpoint crea un nuevo timer con nombre y usuario.
Objetivo	Confirmar que POST /api/timers/ responde con HTTP 201 y devuelve el recurso creado.
Precondiciones	Cuenta autenticada válida.

Datos de entrada	Método: POST<line-break/>URL: https://ps-pruebas.onrender.com/api/timers/<line-break/>Head ers: Content-Type: application/json, Accept: application/json, X-CSRFToken, Cookie<line-break/>Body JSON: name, user
Postcondiciones	Se crea un nuevo registro de timer.
Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 201 Created con JSON que incluye name y user.
Resultado Obtenido	HTTP 201 Created y timer retornado correctamente.
Observaciones	El campo name debe coincidir con el enviado.
Evidencia	
<pre>test('POST timer creates new timer', async ({ page }) => { const csrfToken = await getCsrftoken(page); const response = await page.request.post('/api/timers/', { headers: jsonHeaders(csrfToken), data: { name: 'New fake timer', user: 1, }, }); expect(response.status()).toBe(201); const body = await response.json(); expect(body.name).toBe('New fake timer'); });</pre> ✓ 2 [chromium] > tests\timer.spec.js:15:3 > Timer API > POST timer creates new timer (1.0s)	

1.7.3. TC24 - Validar creación de timer sin usuario explícito (POST)

Campo	Detalle
ID	TC24
Nombre	Validar creación de timer sin usuario explícito
RF asociado	-
Descripción	Verifica que el endpoint crea un timer incluso si no se envía user.
Objetivo	Confirmar que POST /api/timers/ asigna el usuario por defecto y responde con HTTP 201.
Precondiciones	Cuenta autenticada válida.
Datos de entrada	Método: POST<line-break/>URL: https://ps-pruebas.onrender.com/api/timers/<line-break/>Head

	ers: Content-Type: application/json, Accept: application/json, X-CSRFToken, Cookie<line-break/>Body JSON: {} (vacío)
Postcondiciones	Se crea un nuevo registro de timer con usuario por defecto.
Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 201 Created con JSON que incluye user por defecto.
Resultado Obtenido	HTTP 201 Created y timer con user igual a 1.
Observaciones	Verifica comportamiento por defecto del endpoint.
Evidencia	
<pre>test('POST timer default user creates timer', async ({ page }) => { const csrfToken = await getCsrftoken(page); const response = await page.request.post('/api/timers/', { headers: jsonHeaders(csrfToken), data: {}, }); expect(response.status()).toBe(201); const body = await response.json(); expect(body.user).toBe(1); });</pre> ✓ 3 [chromium] > tests\timer.spec.js:29:3 > Timer API > POST timer default user creates timer (1.1s)	

1.7.4. TC25 - Validar actualización de timer (PATCH)

Campo	Detalle
ID	TC25
Nombre	Validar actualización de timer
RF asociado	-
Descripción	Verifica que el endpoint actualiza el nombre de un timer existente.
Objetivo	Confirmar que PATCH /api/timers/{id}/ responde con HTTP 200 y guarda el nuevo nombre.
Precondiciones	Cuenta autenticada válida. Timer existente.
Datos de entrada	Método: PATCH<line-break/>URL: https://ps-pruebas.onrender.com/api/timers/{id}/<line-break/> Headers: Content-Type: application/json, Accept: application/json, X-CSRFToken, Cookie<line-break/>Body JSON: name
Postcondiciones	El timer se actualiza en el sistema.
Prioridad	Media
Estado	Pass

Resultado esperado	HTTP 200 OK con JSON que refleja el nombre actualizado.
Resultado Obtenido	HTTP 200 OK y nombre actualizado correctamente.
Observaciones	Solo se modifica el campo name.
Evidencia	
<pre> test('PATCH timer updates name', async ({ page }) => { const csrfToken = await getCsrfToken(page); const createResponse = await page.request.post('/api/timers/', { headers: jsonHeaders(csrfToken), data: { name: 'Timer to patch', user: 1, }, }); expect(createResponse.status()).toBe(201); const created = await createResponse.json(); const patchResponse = await page.request.patch(`/api/timers/\${created.id}/`, { headers: jsonHeaders(csrfToken), data: { name: 'New Timer Name' }, }); expect(patchResponse.status()).toBe(200); const patched = await patchResponse.json(); expect(patched.name).toBe('New Timer Name'); }); </pre> ✓ 4 [chromium] > tests\timer.spec.js:40:3 > Timer API > PATCH timer updates name (2.0s)	

1.7.5. TC26 - Validar reinicio de timer (PATCH)

Campo	Detalle
ID	TC26
Nombre	Validar reinicio de timer
RF asociado	-
Descripción	Verifica que el endpoint de reinicio funciona repetidamente para un timer existente.
Objetivo	Confirmar que PATCH /api/timers/{id}/restart/ responde con HTTP 200 en múltiples llamadas.
Precondiciones	Cuenta autenticada válida. Timer existente.
Datos de entrada	Método: PATCH<line-break/>URL: https://ps-pruebas.onrender.com/api/timers/{id}/restart/<line-break/>Headers: Accept: application/json, X-CSRFToken, Cookie
Postcondiciones	El timer se reinicia y se puede llamar nuevamente al endpoint.
Prioridad	Media

Estado	Pass
Resultado esperado	Dos llamadas consecutivas devuelven HTTP 200 OK.
Resultado Obtenido	Dos llamadas consecutivas con HTTP 200 OK.
Observaciones	Verifica funcionalidad de reinicio reiterada.
Evidencia	
<pre>test('PATCH timer restart endpoint works twice', async ({ page }) => { const csrfToken = await getCsrfToken(page); const createResponse = await page.request.post('/api/timers/', { headers: jsonHeaders(csrfToken), data: { name: 'Timer restart test', user: 1, }, }); expect(createResponse.status()).toBe(201); const created = await createResponse.json(); const endpoint = `/api/timers/\${created.id}/restart/`; const first = await page.request.patch(endpoint, { headers: jsonHeaders(csrfToken) }); expect(first.status()).toBe(200); const second = await page.request.patch(endpoint, { headers: jsonHeaders(csrfToken) }); expect(second.status()).toBe(200); });</pre> ✓ 5 [chromium] > tests\timer.spec.js:61:3 > Timer API > PATCH timer restart endpoint works twice (1.6s)	

1.8. Sub-Módulo: Tummitime (endpoint /Tummitime/)

1.8.1. TC27 - GET tummy times returns list

Campo	Detalle
ID	TC27
Nombre	Validar listado de tummy time
RF asociado	-
Descripción	Verifica que el endpoint devuelve la lista de registros de tummy time existentes.
Objetivo	Confirmar que GET /api/tummy-times/ responde con HTTP 200 y JSON válido.
Precondiciones	Cuenta autenticada válida. sessionid y csrftoken disponibles.
Datos de entrada	Método: GET<line-break/>URL: https://ps-pruebas.onrender.com/api/tummy-times/<line-break/>Headers: Accept: application/json, Cookie, X-CSRFToken
Postcondiciones	La sesión autenticada permanece activa.
Prioridad	Media
Estado	Pass

Resultado esperado	HTTP 200 OK con JSON que contiene results como arreglo.
Resultado Obtenido	HTTP 200 OK con lista de tummy times.
Observaciones	No requiere body; solo autenticación válida.
Evidencia	
<pre>test('GET tummy times returns list', async ({ page }) => { const csrfToken = await getCsrfToken(page); const response = await page.request.get('/api/tummy-times/', { headers: jsonHeaders(csrfToken), }); expect(response.status()).toBe(200); const body = await response.json(); expect(Array.isArray(body.results)).toBe(true); });</pre> Running 3 tests using 1 worker ✓ 1 [chromium] › tests\tummytime.spec.js:5:3 › Tummy Time API › GET tummy times returns list (1.2s)	

1.8.2. TC28 - Validar creación de tummy time con duración (POST)

Campo	Detalle
ID	TC28
Nombre	Validar creación de tummy time con duración
RF asociado	-
Descripción	Verifica que el endpoint crea un registro de tummy time y calcula la duración.
Objetivo	Confirmar que POST /api/tummy-times/ responde con HTTP 201 y devuelve duration.
Precondiciones	Cuenta autenticada válida. child válido existente.
Datos de entrada	M"etodo: POST<line-break/>URL: https://ps-pruebas.onrender.com/api/tummy-times/<line-break/>Headers: Content-Type: application/json, Accept: application/json, X-CSRFToken, Cookie<line-break/>Body JSON: child, start, end, milestone
Postcondiciones	Se crea un nuevo registro de tummy time con duración calculada.
Prioridad	Media
Estado	Pass

Resultado esperado	HTTP 201 Created con JSON que incluye duration calculada.
Resultado Obtenido	HTTP 201 Created y duración correcta.
Observaciones	La duración debe coincidir con la diferencia entre start y end.
Evidencia	
<pre> test('POST tummy time creates entry with duration', async ({ page }) => { const childId = await createChild(page); const csrfToken = await getCsrftoken(page); const response = await page.request.post('/api/tummy-times/', { headers: jsonHeaders(csrfToken), data: { child: childId, start: '2017-11-18T12:30:00-05:00', end: '2017-11-18T12:35:30-05:00', milestone: 'Rolled over.', }, }); expect(response.status()).toBe(201); const body = await response.json(); expect(body.duration).toMatch(/0:05:30 00:05:30/); }); </pre> ✓ 2 [chromium] > tests\tummytime.spec.js:15:3 > Tummy Time API > POST tummy time creates entry with duration (1.8s)	

1.8.3. TC29 - Validar actualización de milestone de tummy time (PATCH)

Campo	Detalle
ID	TC29
Nombre	Validar actualización de milestone de tummy time
RF asociado	-
Descripción	Verifica que el endpoint actualiza el campo milestone de un tummy time existente.
Objetivo	Confirmar que PATCH /api/tummy-times/{id}/ responde con HTTP 200 y devuelve el milestone actualizado.
Precondiciones	Cuenta autenticada válida. Registro de tummy time existente.
Datos de entrada	Método: PATCH<line-break/>URL: https://ps-pruebas.onrender.com/api/tummy-times/{id}/<line-break/>Headers: Content-Type: application/json, Accept: application/json, X-CSRFToken, Cookie<line-break/>Body JSON: milestone

Postcondiciones	El registro de tummy time se actualiza con el nuevo milestone.
Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 200 OK con JSON que muestra milestone actualizado.
Resultado Obtenido	HTTP 200 OK y milestone actualizado correctamente.
Observaciones	Solo se modifica el campo milestone.
Evidencia	
<pre> test('PATCH tummy time updates milestone', async ({ page }) => { const childId = await createChild(page); const csrfToken = await getCsrfToken(page); const createResponse = await page.request.post('/api/tummy-times/', { headers: jsonHeaders(csrfToken), data: { child: childId, start: '2017-11-18T15:30:00-05:00', end: '2017-11-18T15:30:45-05:00', milestone: '', }, }); expect(createResponse.status()).toBe(201); const created = await createResponse.json(); const patchResponse = await page.request.patch(`/api/tummy-times/\${created.id}/`, { headers: jsonHeaders(csrfToken), data: { milestone: 'Switched sides!' }, }); expect(patchResponse.status()).toBe(200); const patched = await patchResponse.json(); expect(patched.milestone).toBe('Switched sides!'); }); </pre> ✓ 3 [chromium] › tests\tummytime.spec.js:32:3 › Tummy Time API › PATCH tummy time updates milestone (2.0s)	

1.9. Sub-Módulo: Weight (endpoint /Weight/)

1.9.1. TC30 - Validar listado de peso (GET)

Campo	Detalle
ID	TC30
Nombre	Validar listado de peso
RF asociado	-
Descripción	Verifica que el endpoint devuelve la lista de registros de peso existentes.
Objetivo	Confirmar que GET /api/weight/ responde con HTTP 200 y JSON válido.

Precondiciones	Cuenta autenticada válida. sessionid y csrftoken disponibles.
Datos de entrada	Método: GET<line-break/>URL: https://ps-pruebas.onrender.com/api/weight/<line-break/>Hea ders: Accept: application/json, Cookie, X-CSRFToken
Postcondiciones	La sesión autenticada permanece activa.
Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 200 OK con JSON que contiene results como arreglo de pesos.
Resultado Obtenido	HTTP 200 OK con lista de registros de peso.
Observaciones	No requiere body; solo autenticación válida.
Evidencia	
<pre>test('GET weight returns list', async ({ page }) => { const csrfToken = await getCsrfToken(page); const response = await page.request.get('/api/weight/', { headers: jsonHeaders(csrfToken), }); expect(response.status()).toBe(200); const body = await response.json(); expect(Array.isArray(body.results)).toBe(true); });</pre> Running 4 tests using 1 worker ✓ 1 [chromium] > tests\weight.spec.js:5:3 > Weight API > GET weight returns list (1.5s)	

1.9.2. TC31 - Validar creación de peso con fecha explícita (POST)

Campo	Detalle
ID	TC31
Nombre	Validar creación de peso con fecha explícita
RF asociado	-
Descripción	Verifica que el endpoint crea un registro de peso usando la fecha proporcionada.
Objetivo	Confirmar que POST /api/weight/ responde con HTTP 201 y devuelve el peso con la fecha enviada.
Precondiciones	Cuenta autenticada válida. child válido existente.

Datos de entrada	Método: POST<line-break/>URL: https://ps-pruebas.onrender.com/api/weight/<line-break/>Hea ders: Content-Type: application/json, Accept: application/json, X-CSRFToken, Cookie<line-break/>Body JSON: child, weight, date, notes
Postcondiciones	Se crea un nuevo registro de peso con la fecha indicada.
Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 201 Created con JSON que incluye weight numérico y date.
Resultado Obtenido	HTTP 201 Created y peso retornado correctamente.
Observaciones	El valor weight se espera como número en la respuesta.
Evidencia	
<pre>test('POST weight with date explicitly', async ({ page }) => { const childId = await createChild(page); const csrfToken = await getCsrfToken(page); const response = await page.request.post('/api/weight/', { headers: jsonHeaders(csrfToken), data: { child: childId, weight: '9.75', date: '2017-11-20', notes: 'after feed', }, }); expect(response.status()).toBe(201); const body = await response.json(); expect(body.weight).toBe(9.75); });</pre> ✓ 2 [chromium] > tests\weight.spec.js:15:3 > Weight API > POST weight with date explicitly (1.7s)	

1.9.3. TC32 - Validar creación de peso sin fecha (POST)

Campo	Detalle
ID	TC32
Nombre	Validar creación de peso sin fecha

RF asociado	-
Descripción	Verifica que el endpoint crea un registro de peso cuando no se envía fecha.
Objetivo	Confirmar que POST /api/weight/ responde con HTTP 201 y asigna fecha actual.
Precondiciones	Cuenta autenticada válida. child válido existente.
Datos de entrada	Método: POST<line-break/>URL: https://ps-pruebas.onrender.com/api/weight/<line-break/>Headers: Content-Type: application/json, Accept: application/json, X-CSRFToken, Cookie<line-break/>Body JSON: child, weight, notes
Postcondiciones	Se crea un nuevo registro de peso con fecha por defecto.
Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 201 Created con JSON que incluye weight numérico y fecha calculada.
Resultado Obtenido	HTTP 201 Created y peso retornado correctamente.
Observaciones	La fecha puede establecerse automáticamente en el servidor.
Evidencia	
<pre>test('POST weight without date uses current date', async ({ page }) => { const childId = await createChild(page); const csrfToken = await getCsrfToken(page); const response = await page.request.post('/api/weight/', { headers: jsonHeaders(csrfToken), data: { child: childId, weight: '12.25', notes: 'with diaper at peds', }, }); expect(response.status()).toBe(201); const body = await response.json(); expect(body.weight).toBe(12.25); });</pre> ✓ 3 [chromium] › tests\weight.spec.js:32:3 › Weight API › POST weight without date uses current date (1.9s)	

1.9.4. TC33 - Validar actualización de peso (PATCH)

Campo	Detalle
-------	---------

ID	TC33
Nombre	Validar actualización de peso
RF asociado	-
Descripción	Verifica que el endpoint actualiza el valor de peso de un registro existente.
Objetivo	Confirmar que PATCH /api/weight/{id}/ responde con HTTP 200 y devuelve el nuevo peso.
Precondiciones	Cuenta autenticada válida. Registro de peso existente.
Datos de entrada	Método: PATCH<line-break/>URL: https://ps-pruebas.onrender.com/api/weight/{id}/<line-break/> Headers: Content-Type: application/json, Accept: application/json, X-CSRFToken, Cookie<line-break/>Body JSON: weight
Postcondiciones	El registro de peso se actualiza en el sistema.
Prioridad	Media
Estado	Pass
Resultado esperado	HTTP 200 OK con JSON que muestra weight actualizado.
Resultado Obtenido	HTTP 200 OK y peso actualizado correctamente.
Observaciones	El valor weight se envía como número y debe devolverse como número.
Evidencia	

```

test('PATCH weight updates value', async ({ page }) => {
  const childId = await createChild(page);
  const csrfToken = await getCsrfToken(page);
  const createResponse = await page.request.post('/api/weight/', {
    headers: jsonHeaders(csrfToken),
    data: {
      child: childId,
      weight: '9.5',
      date: '2017-11-18',
      notes: 'before feed',
    },
  });
  expect(createResponse.status()).toBe(201);
  const created = await createResponse.json();

  const patchResponse = await page.request.patch(`/api/weight/${created.id}/`, {
    headers: jsonHeaders(csrfToken),
    data: {
      weight: '8.25',
    },
  });
  expect(patchResponse.status()).toBe(200);
  const patched = await patchResponse.json();
  expect(patched.weight).toBe(8.25);
});

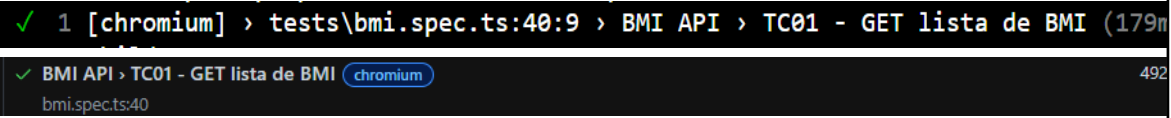
```

✓ 4 [chromium] > tests\weight.spec.js:48:3 > Weight API > PATCH weight updates value (1.9s)

1.10. Sub-Módulo: BMI (endpoint /bmi/)

1.10.1. TC34 — Validar listado de BMI (GET)

Campo	Detalle
ID	TC34
Nombre	Validar listado de BMI
RF asociado	RF-004.8
Descripción	Verifica que el endpoint devuelve la lista de registros de BMI.
Objetivo	Confirmar que GET /bmi/ responde con HTTP 200 y un JSON con la clave "results" como arreglo.
Precondiciones	Cuenta autenticada válida (Token obtenido en /user/settings/). Header Authorization: Token <API_KEY>.
Datos de entrada	Método: GET URL: https://ps-pruebas.onrender.com/api/bmi/ Headers: Authorization: Token <API_KEY>, Accept: application/json
Postcondiciones	La sesión autenticada permanece activa. No se modifican datos.

Prioridad	Media
Estado	pass
Resultado esperado	HTTP 200 OK con JSON que contiene "results" como arreglo.
Resultado Obtenido	HTTP 200 OK con JSON que contiene "results" como arreglo.
Observaciones	No requiere body; solo autenticación válida.
Evidencia	
<pre>test('TC01 - GET lista de BMI', async () => { const res = await api.get('bmi/'); expect(res.status()).toBe(200); const body = await res.json(); expect(Array.isArray(body.results)).toBeTruthy(); });</pre> 	

1.10.2. TC35 — Validar creación de registro de BMI (POST)

Campo	Detalle
ID	TC35
Nombre	Validar creación de registro de BMI
RF asociado	RF-004.8
Descripción	Verifica que el endpoint crea un nuevo registro de BMI con datos válidos.
Objetivo	Confirmar que POST /bmi/ responde con HTTP 201 y devuelve el objeto creado.
Precondiciones	Cuenta autenticada válida (Token obtenido en /user/settings/). Header Authorization: Token <API_KEY>. Debe existir al menos un "child" válido (id=1) en el sistema, excepto para el módulo Child.
Datos de entrada	Método: POST URL: https://ps-pruebas.onrender.com/api/bmi/ Headers: Authorization: Token <API_KEY>, Content-Type: application/json

	Body JSON (child, bmi, date, notes): {"child": 1, "bmi": "18.5", "date": "2026-07-05", "notes": "QA_TEST"}
Postcondiciones	Se crea un nuevo registro de BMI identificable por el prefijo "QA_TEST" para su posterior limpieza.
Prioridad	Alta
Estado	pass
Resultado esperado	HTTP 201 Created con JSON del objeto creado, incluyendo su id.
Resultado Obtenido	HTTP 201 Created con JSON del objeto creado, incluyendo su id.
Observaciones	Guardar el id devuelto; se usará en los TC de PATCH y DELETE de este módulo.
Evidencia	
<pre>test('TC02 - POST crear registro de BMI', async () => { const res = await api.post('bmi/', { data: { child: testChildId, bmi: '18.5', date: '2026-07-05', notes: 'QA_TEST' }, }); console.log('URL pedida:', res.url()); console.log('Status:', res.status()); console.log('Body:', await res.text()); createdId = (await res.json()).id; });</pre> ✓ 2 [chromium] > tests\bmi.spec.ts:47:9 > BMI API > TC02 - POST crear registro de BMI (476ms) ✓ BMI API > TC02 - POST crear registro de BMI (chromium) 215ms bmi.spec.ts:47	

1.10.3. TC36 — Validar actualización parcial de BMI (PATCH)

Campo	Detalle
ID	TC36
Nombre	Validar actualización parcial de BMI
RF asociado	RF-004.8
Descripción	Verifica que el endpoint actualiza parcialmente un registro existente de BMI.
Objetivo	Confirmar que PATCH /bmi/{id}/ responde con HTTP 200 y refleja el cambio solicitado.
Precondiciones	Debe existir el registro creado en el TC de POST anterior (mismo id).

Datos de entrada	Método: PATCH URL: https://ps-pruebas.onrender.com/api/bmi/{id}/ Headers: Authorization: Token <API_KEY>, Content-Type: application/json Body JSON: { "bmi": "19.0" }
Postcondiciones	El registro queda modificado con el nuevo valor.
Prioridad	Media
Estado	pass
Resultado esperado	HTTP 200 OK con JSON que refleja el campo actualizado.
Resultado Obtenido	HTTP 200 OK con JSON que refleja el campo actualizado.
Observaciones	Verificar con un GET adicional que el cambio persiste.
Evidencia	
<pre>test('TC03 - PATCH actualizar registro de BMI', async () => { expect(createdId).toBeTruthy(); const res = await api.patch(`bmi/\${createdId}/`, { data: { bmi: '19.0' }, }); expect(res.status()).toBe(200); const body = await res.json(); expect(body.bmi).toBe(19.0); });</pre> ✓ 3 [chromium] > tests\bmi.spec.ts:57:9 > BMI API > TC03 - PATCH actualizar registro de BMI (250ms) ✓ BMI API > TC03 - PATCH actualizar registro de BMI chromium 175ms bmi.spec.ts:57	

1.10.4. TC37 — Validar eliminación de registro de BMI (DELETE)

Campo	Detalle
ID	TC37
Nombre	Validar eliminación de registro de BMI
RF asociado	RF-004.8
Descripción	Verifica que el endpoint elimina un registro existente de BMI.
Objetivo	Confirmar que DELETE /bmi/{id}/ responde con HTTP 204 y que el recurso deja de existir.

Precondiciones	Debe existir el registro creado en el TC de POST de este módulo (mismo id).
Datos de entrada	Método: DELETE URL: https://ps-pruebas.onrender.com/api/bmi/{id}/ Headers: Authorization: Token <API_KEY>
Postcondiciones	El registro de prueba queda eliminado; el sistema vuelve al estado previo a la ejecución (limpieza).
Prioridad	Media
Estado	pass
Resultado esperado	HTTP 204 No Content. Un GET posterior al mismo id retorna HTTP 404.
Resultado Obtenido	HTTP 204 No Content. Un GET posterior al mismo id retorna HTTP 404.
Observaciones	Este paso cumple el rol de limpieza (teardown) del dato creado en el POST.
Evidencia	
<pre>test('TC04 - DELETE eliminar registro de BMI', async () => { expect(createdId).toBeTruthy(); const res = await api.delete(`bmi/\${createdId}/`); expect(res.status()).toBe(204); const check = await api.get(`bmi/\${createdId}/`); expect(check.status()).toBe(404); });</pre> ✓ 4 [chromium] > tests\bmi.spec.ts:67:9 > BMI API > TC04 - DELETE eliminar registro de BMI (355m) ✓ BMI API > TC04 - DELETE eliminar registro de BMI (chromium) 373 bmi.spec.ts:67	

1.11. Sub-Módulo: Child (endpoint /children/)

1.11.1. TC38 — Validar listado de Child (GET)

Campo	Detalle
ID	TC38
Nombre	Validar listado de Child
RF asociado	RF-003.1, RF-003.2

Descripción	Verifica que el endpoint devuelve la lista de registros de Child.
Objetivo	Confirmar que GET /children/ responde con HTTP 200 y un JSON con la clave "results" como arreglo.
Precondiciones	Cuenta autenticada válida (Token obtenido en /user/settings/). Header Authorization: Token <API_KEY>. Se identifica por slug, no por ID numérico.
Datos de entrada	Método: GET URL: https://ps-pruebas.onrender.com/api/children/ Headers: Authorization: Token <API_KEY>, Accept: application/json
Postcondiciones	La sesión autenticada permanece activa. No se modifican datos.
Prioridad	Media
Estado	pass
Resultado esperado	HTTP 200 OK con JSON que contiene "results" como arreglo.
Resultado Obtenido	HTTP 200 OK con JSON que contiene "results" como arreglo.
Observaciones	No requiere body; solo autenticación válida.
Evidencia	
<pre>test('TC01 - GET children/ devuelve lista de niños', async () => { const response = await api.get('children/'); expect(response.status()).toBe(200); const body = await response.json(); expect(Array.isArray(body.results)).toBeTruthy(); });</pre> ✓ 1 ...um] > tests\child.spec.ts:32:9 > Child - Pruebas de Integración > TC01 - GET children/ devuelve lista de niños (682ms) ✓ Child - Pruebas de Integración > TC01 - GET children/ devuelve lista de niños chromium 682ms child.specs:32	

1.11.2. TC39 — Validar creación de registro de Child (POST)

Campo	Detalle
ID	TC39
Nombre	Validar creación de registro de Child
RF asociado	RF-003.1, RF-003.2

Descripción	Verifica que el endpoint crea un nuevo registro de Child con datos válidos.
Objetivo	Confirmar que POST /children/ responde con HTTP 201 y devuelve el objeto creado.
Precondiciones	Cuenta autenticada válida (Token obtenido en /user/settings/). Header Authorization: Token <API_KEY>. Debe existir al menos un "child" válido (id=1) en el sistema, excepto para el módulo Child.
Datos de entrada	Método: POST URL: https://ps-pruebas.onrender.com/api/children/ Headers: Authorization: Token <API_KEY>, Content-Type: application/json Body JSON (first_name, last_name, birth_date, birth_time, slug): { "first_name": "QA", "last_name": "Test", "birth_date": "2026-01-01" }
Postcondiciones	Se crea un nuevo registro de Child identificable por el prefijo "QA_TEST" para su posterior limpieza.
Prioridad	Alta
Estado	pass
Resultado esperado	HTTP 201 Created con JSON del objeto creado, incluyendo su id.
Resultado Obtenido	HTTP 201 Created con JSON del objeto creado, incluyendo su id.
Observaciones	Guardar el id devuelto; se usará en los TC de PATCH y DELETE de este módulo.
Evidencia	

```
test('TC02 - POST children/ crea un nuevo niño', async () => {
  const data = {
    first_name: 'QA',
    last_name: 'Playwright',
    birth_date: '2024-01-01',
    birth_time: '10:00',
  };

  const response = await api.post('children/', { data });
  expect(response.status()).toBe(201);

  const body = await response.json();
  expect(body.first_name).toBe(data.first_name);
  expect(body.last_name).toBe(data.last_name);
  expect(body.slug).toBeTruthy();

  createdSlug = body.slug;
});
```

✓ 2 ...romium] > tests\child.spec.ts:40:9 > Child - Pruebas de Integración > TC02 - POST children/ crea un nuevo niño (218ms)

✓ Child - Pruebas de Integración > TC02 - POST children/ crea un nuevo niño [chromium](#) 218ms

child.spec.ts:40

1.11.3. TC40 — Validar actualización parcial de Child (PATCH)

Campo	Detalle
ID	TC40
Nombre	Validar actualización parcial de Child
RF asociado	RF-003.1, RF-003.2
Descripción	Verifica que el endpoint actualiza parcialmente un registro existente de Child.
Objetivo	Confirmar que PATCH /children/{id}/ responde con HTTP 200 y refleja el cambio solicitado.
Precondiciones	Debe existir el registro creado en el TC de POST anterior (mismo id).
Datos de entrada	Método: PATCH URL: https://ps-pruebas.onrender.com/api/children/{id}/ Headers: Authorization: Token <API_KEY>, Content-Type: application/json Body JSON: { "first_name": "QA_Updated" }

Postcondiciones	El registro queda modificado con el nuevo valor.
Prioridad	Media
Estado	pass
Resultado esperado	HTTP 200 OK con JSON que refleja el campo actualizado.
Resultado Obtenido	HTTP 200 OK con JSON que refleja el campo actualizado.
Observaciones	Verificar con un GET adicional que el cambio persiste.
Evidencia	
<pre>test('TC03 - PATCH children/{slug}/ actualiza el nombre del niño', async () => { const response = await api.patch(`children/\${createdSlug}/`, { data: { first_name: 'QAUpdated', last_name: 'PlaywrightUpdated', }, }); expect(response.status()).toBe(200); const body = await response.json(); expect(body.first_name).toBe('QAUpdated'); expect(body.last_name).toBe('PlaywrightUpdated'); // El slug puede cambiar al actualizar el nombre: se recaptura para el DELETE createdSlug = body.slug; });</pre> ✓ 3 ...ild.spec.ts:59:9 > Child - Pruebas de Integración > TC03 - PATCH children/{slug}/ actualiza el nombre del niño (192ms) ✓ Child - Pruebas de Integración > TC03 - PATCH children/{slug}/ actualiza el nombre del niño chromium 192ms child.spec.ts:59	

1.11.4. TC41 — Validar eliminación de registro de Child (DELETE)

Campo	Detalle
ID	TC41
Nombre	Validar eliminación de registro de Child
RF asociado	RF-003.1, RF-003.2
Descripción	Verifica que el endpoint elimina un registro existente de Child.
Objetivo	Confirmar que DELETE /children/{id}/ responde con HTTP 204 y que el recurso deja de existir.
Precondiciones	Debe existir el registro creado en el TC de POST de este módulo (mismo id).
Datos de entrada	Método: DELETE

	URL: https://ps-pruebas.onrender.com/api/children/{id}/ Headers: Authorization: Token <API_KEY>
Postcondiciones	El registro de prueba queda eliminado; el sistema vuelve al estado previo a la ejecución (limpieza).
Prioridad	Media
Estado	pass
Resultado esperado	HTTP 204 No Content. Un GET posterior al mismo id retorna HTTP 404.
Resultado Obtenido	HTTP 204 No Content. Un GET posterior al mismo id retorna HTTP 404.
Observaciones	Este paso cumple el rol de limpieza (teardown) del dato creado en el POST.
Evidencia	
<pre>test('TC04 - DELETE children/{slug}/ elimina el niño creado', async () => { const response = await api.delete(`children/\${createdSlug}/`); expect(response.status()).toBe(204); // Verificación: ya no debe encontrarse const verify = await api.get(`children/\${createdSlug}/`); expect(verify.status()).toBe(404); });</pre> ✓ 4 ...ts\child.spec.ts:76:9 > Child - Pruebas de Integración > TC04 - DELETE children/{slug}/ elimina el niño creado (394ms) ✓ Child - Pruebas de Integración > TC04 - DELETE children/{slug}/ elimina el niño creado chromium 394ms child.spec.ts:76	

1.12. Sub-Módulo: Pumping (endpoint /pumping/)

1.12.1. TC42 — Validar listado de Pumping (GET)

Campo	Detalle
ID	TC42
Nombre	Validar listado de Pumping
RF asociado	RF-004.5, RF-004.12
Descripción	Verifica que el endpoint devuelve la lista de registros de Pumping.
Objetivo	Confirmar que GET /pumping/ responde con HTTP 200 y un JSON con la clave "results" como arreglo.
Precondiciones	Cuenta autenticada válida (Token obtenido en /user/settings/). Header Authorization: Token <API_KEY>.

Datos de entrada	Método: GET URL: https://ps-pruebas.onrender.com/api/pumping/ Headers: Authorization: Token <API_KEY>, Accept: application/json
Postcondiciones	La sesión autenticada permanece activa. No se modifican datos.
Prioridad	Media
Estado	pass
Resultado esperado	HTTP 200 OK con JSON que contiene "results" como arreglo.
Resultado Obtenido	HTTP 200 OK con JSON que contiene "results" como arreglo.
Observaciones	No requiere body; solo autenticación válida.
Evidencia	
<pre>test('TC01 - GET pumping/ devuelve lista de extracciones', async () => { const response = await api.get('pumping/'); expect(response.status()).toBe(200); const body = await response.json(); expect(Array.isArray(body.results)).toBeTruthy(); });</pre> ✓ 1 ...s\pumping.spec.ts:45:9 > Pumping - Pruebas de Integración > TC01 - GET pumping/ devuelve lista de extracciones (195ms) ✓ Pumping - Pruebas de Integración > TC01 - GET pumping/ devuelve lista de extracciones chromium 195ms pumping.spec.ts:45	

1.12.2. TC43 — Validar creación de registro de Pumping (POST)

Campo	Detalle
ID	TC43
Nombre	Validar creación de registro de Pumping
RF asociado	RF-004.5, RF-004.12
Descripción	Verifica que el endpoint crea un nuevo registro de Pumping con datos válidos.
Objetivo	Confirmar que POST /pumping/ responde con HTTP 201 y devuelve el objeto creado.
Precondiciones	Cuenta autenticada válida (Token obtenido en /user/settings/). Header Authorization: Token <API_KEY>. Debe existir al menos un "child" válido (id=1) en el sistema, excepto para el módulo Child.

Datos de entrada	Método: POST URL: https://ps-pruebas.onrender.com/api/pumping/ Headers: Authorization: Token <API_KEY>, Content-Type: application/json Body JSON (child, amount, start, end, notes): { "child": 1, "amount": "20.0", "start": "2026-07-05T10:00:00-05:00", "end": "2026-07-05T10:15:00-05:00", "notes": "QA_TEST" }
Postcondiciones	Se crea un nuevo registro de Pumping identificable por el prefijo "QA_TEST" para su posterior limpieza.
Prioridad	Alta
Estado	pass
Resultado esperado	HTTP 201 Created con JSON del objeto creado, incluyendo su id.
Resultado Obtenido	HTTP 201 Created con JSON del objeto creado, incluyendo su id.
Observaciones	Guardar el id devuelto; se usará en los TC de PATCH y DELETE de este módulo.
Evidencia	
<pre>test('TC02 - POST pumping/ crea un nuevo registro de extracción', async () => { const data = { child: testChildId, amount: '21.0', start: '2026-07-01T22:52:00-05:00', end: '2026-07-01T23:05:00-05:00', notes: 'old device', }; const response = await api.post('pumping/', { data }); expect(response.status()).toBe(201); const body = await response.json(); // amount se envía como string, el servidor lo devuelve como number expect(body.amount).toBe(21.0); expect(body.notes).toBe(data.notes); createdId = body.id; });</pre> ✓ 2 ...ng.spec.ts:53:9 › Pumping - Pruebas de Integración › TC02 - POST pumping/ crea un nuevo registro de extracción (497ms) ✓ Pumping - Pruebas de Integración › TC02 - POST pumping/ crea un nuevo registro de extracción chromium 497ms pumping.spec.ts:53	

1.12.3. TC44 — Validar actualización parcial de Pumping (PATCH)

Campo	Detalle
ID	TC44
Nombre	Validar actualización parcial de Pumping
RF asociado	RF-004.5, RF-004.12
Descripción	Verifica que el endpoint actualiza parcialmente un registro existente de Pumping.
Objetivo	Confirmar que PATCH /pumping/{id}/ responde con HTTP 200 y refleja el cambio solicitado.
Precondiciones	Debe existir el registro creado en el TC de POST anterior (mismo id).
Datos de entrada	Método: PATCH URL: https://ps-pruebas.onrender.com/api/pumping/{id}/ Headers: Authorization: Token <API_KEY>, Content-Type: application/json Body JSON: {"amount": "25.0"}
Postcondiciones	El registro queda modificado con el nuevo valor.
Prioridad	Media
Estado	pass
Resultado esperado	HTTP 200 OK con JSON que refleja el campo actualizado.
Resultado Obtenido	HTTP 200 OK con JSON que refleja el campo actualizado.
Observaciones	Verificar con un GET adicional que el cambio persiste.
Evidencia	
<pre>test('TC03 - PATCH pumping/{id}/ actualiza el amount', async () => { const response = await api.patch(`pumping/\${createdId}/`, { data: { amount: 41 }, }); expect(response.status()).toBe(200); const body = await response.json(); expect(body.amount).toBe(41); });</pre> ✓ 3 ..tests\pumping.spec.ts:73:9 › Pumping - Pruebas de Integración › TC03 - PATCH pumping/{id}/ actualiza el amount (183ms) ✓ Pumping - Pruebas de Integración › TC03 - PATCH pumping/{id}/ actualiza el amount chromium 183ms pumping.spec.ts:73	

1.12.4. TC45 — Validar eliminación de registro de Pumping (DELETE)

Campo	Detalle
ID	TC45
Nombre	Validar eliminación de registro de Pumping
RF asociado	RF-004.5, RF-004.12
Descripción	Verifica que el endpoint elimina un registro existente de Pumping.
Objetivo	Confirmar que DELETE /pumping/{id}/ responde con HTTP 204 y que el recurso deja de existir.
Precondiciones	Debe existir el registro creado en el TC de POST de este módulo (mismo id).
Datos de entrada	Método: DELETE URL: https://ps-pruebas.onrender.com/api/pumping/{id}/ Headers: Authorization: Token <API_KEY>
Postcondiciones	El registro de prueba queda eliminado; el sistema vuelve al estado previo a la ejecución (limpieza).
Prioridad	Media
Estado	pass
Resultado esperado	HTTP 204 No Content. Un GET posterior al mismo id retorna HTTP 404.
Resultado Obtenido	HTTP 204 No Content. Un GET posterior al mismo id retorna HTTP 404.
Observaciones	Este paso cumple el rol de limpieza (teardown) del dato creado en el POST.
Evidencia	
<pre>test('TC04 - DELETE pumping/{id}/ elimina el registro creado', async () => { const response = await api.delete(`pumping/\${createdId}/`); expect(response.status()).toBe(204); const verify = await api.get(`pumping/\${createdId}/`); expect(verify.status()).toBe(404); });</pre>	
✓ 4 ...mping.spec.ts:83:9 › Pumping - Pruebas de Integración › TC04 - DELETE pumping/{id}/ elimina el registro creado (386ms)	
✓ Pumping - Pruebas de Integración › TC04 - DELETE pumping/{id}/ elimina el registro creado chromium 386ms pumping.spec.ts:83	

1.13. Sub-Módulo: DiaperChange (endpoint /changes/)

1.13.1. TC46 — Validar listado de DiaperChange (GET)

Campo	Detalle
ID	TC46
Nombre	Validar listado de DiaperChange
RF asociado	RF-004.6, RF-006.1
Descripción	Verifica que el endpoint devuelve la lista de registros de DiaperChange.
Objetivo	Confirmar que GET /changes/ responde con HTTP 200 y un JSON con la clave "results" como arreglo.
Precondiciones	Cuenta autenticada válida (Token obtenido en /user/settings/). Header Authorization: Token <API_KEY>.
Datos de entrada	Método: GET URL: https://ps-pruebas.onrender.com/api/changes/ Headers: Authorization: Token <API_KEY>, Accept: application/json
Postcondiciones	La sesión autenticada permanece activa. No se modifican datos.
Prioridad	Media
Estado	pass
Resultado esperado	HTTP 200 OK con JSON que contiene "results" como arreglo.
Resultado Obtenido	HTTP 200 OK con JSON que contiene "results" como arreglo.
Observaciones	No requiere body; solo autenticación válida.
Evidencia	
<pre>test('TC01 - GET changes/ devuelve lista de cambios de pañal', async () => { const response = await api.get('changes/'); expect(response.status()).toBe(200); const body = await response.json(); expect(Array.isArray(body.results)).toBeTruthy(); });</pre>	
✓ 1 ...hange - Pruebas de Integración > TC01 - GET changes/ devuelve lista de cambios de pañal (197ms)	
✓ DiaperChange - Pruebas de Integración > TC01 - GET changes/ devuelve lista de cambios de pañal chromium 197ms diaperChange.spec.ts:44	

1.13.2. TC47 — Validar creación de registro de DiaperChange (POST)

Campo	Detalle
ID	TC47
Nombre	Validar creación de registro de DiaperChange
RF asociado	RF-004.6, RF-006.1
Descripción	Verifica que el endpoint crea un nuevo registro de DiaperChange con datos válidos.
Objetivo	Confirmar que POST /changes/ responde con HTTP 201 y devuelve el objeto creado.
Precondiciones	Cuenta autenticada válida (Token obtenido en /user/settings/). Header Authorization: Token <API_KEY>. Debe existir al menos un "child" válido (id=1) en el sistema, excepto para el módulo Child.
Datos de entrada	Método: POST URL: https://ps-pruebas.onrender.com/api/changes/ Headers: Authorization: Token <API_KEY>, Content-Type: application/json Body JSON (child, time, wet, solid, color, amount, notes): { "child": 1, "time": "2026-07-05T10:00:00-05:00", "wet": true, "solid": false, "color": "", "notes": "QA_TEST" }
Postcondiciones	Se crea un nuevo registro de DiaperChange identificable por el prefijo "QA_TEST" para su posterior limpieza.
Prioridad	Alta
Estado	pass
Resultado esperado	HTTP 201 Created con JSON del objeto creado, incluyendo su id.
Resultado Obtenido	HTTP 201 Created con JSON del objeto creado, incluyendo su id.
Observaciones	Guardar el id devuelto; se usará en los TC de PATCH y DELETE de este módulo.
Evidencia	

```
test('TC02 - POST changes/ crea un nuevo cambio de pañal', async () => {
  const data = {
    child: testChildId,
    time: '2026-07-01T12:00:00-05:00',
    wet: true,
    solid: true,
    color: 'brown',
    amount: 1.25,
    notes: 'seedy',
  };

  const response = await api.post('changes/', { data });
  expect(response.status()).toBe(201);

  const body = await response.json();
  expect(body.wet).toBe(true);
  expect(body.solid).toBe(true);
  expect(body.color).toBe(data.color);
  expect(body.amount).toBe(data.amount);
  expect(body.notes).toBe(data.notes);

  const createdId = body.id;
});
```

✓ 2 ...perChange - Pruebas de Integración > TC02 - POST changes/ crea un nuevo cambio de pañal (175ms)

✓ DiaperChange - Pruebas de Integración > TC02 - POST changes/ crea un nuevo cambio de pañal chromium 175ms
diaperChange.spects:52

1.13.3. TC48 — Validar actualización parcial de DiaperChange (PATCH)

Campo	Detalle
ID	TC48
Nombre	Validar actualización parcial de DiaperChange
RF asociado	RF-004.6, RF-006.1
Descripción	Verifica que el endpoint actualiza parcialmente un registro existente de DiaperChange.
Objetivo	Confirmar que PATCH /changes/{id}/ responde con HTTP 200 y refleja el cambio solicitado.
Precondiciones	Debe existir el registro creado en el TC de POST anterior (mismo id).
Datos de entrada	Método: PATCH URL: https://ps-pruebas.onrender.com/api/changes/{id}/

	Headers: Authorization: Token <API_KEY>, Content-Type: application/json Body JSON: {"wet": false, "solid": true}
Postcondiciones	El registro queda modificado con el nuevo valor.
Prioridad	Media
Estado	pass
Resultado esperado	HTTP 200 OK con JSON que refleja el campo actualizado.
Resultado Obtenido	HTTP 200 OK con JSON que refleja el campo actualizado.
Observaciones	Verificar con un GET adicional que el cambio persiste.
Evidencia	
<pre>test('TC03 - PATCH changes/{id}/ actualiza wet y solid', async () => { const response = await api.patch(`changes/\${createdId}/`, { data: { wet: false, solid: true }, }); expect(response.status()).toBe(200); const body = await response.json(); expect(body.wet).toBe(false); expect(body.solid).toBe(true); });</pre> ✓ 3 ...iaperChange - Pruebas de Integración > TC03 - PATCH changes/{id}/ actualiza wet y solid (176ms) ✓ DiaperChange - Pruebas de Integración > TC03 - PATCH changes/{id}/ actualiza wet y solid chromium 176ms diaperChange.spects:76	

1.13.4. TC49 — Validar eliminación de registro de DiaperChange (DELETE)

Campo	Detalle
ID	TC49
Nombre	Validar eliminación de registro de DiaperChange
RF asociado	RF-004.6, RF-006.1
Descripción	Verifica que el endpoint elimina un registro existente de DiaperChange.
Objetivo	Confirmar que DELETE /changes/{id}/ responde con HTTP 204 y que el recurso deja de existir.

Precondiciones	Debe existir el registro creado en el TC de POST de este módulo (mismo id).
Datos de entrada	Método: DELETE URL: https://ps-pruebas.onrender.com/api/changes/{id}/ Headers: Authorization: Token <API_KEY>
Postcondiciones	El registro de prueba queda eliminado; el sistema vuelve al estado previo a la ejecución (limpieza).
Prioridad	Media
Estado	pass
Resultado esperado	HTTP 204 No Content. Un GET posterior al mismo id retorna HTTP 404.
Resultado Obtenido	HTTP 204 No Content. Un GET posterior al mismo id retorna HTTP 404.
Observaciones	Este paso cumple el rol de limpieza (teardown) del dato creado en el POST.
Evidencia	
<pre>test('TC04 - DELETE changes/{id}/ elimina el registro creado', async () => { const response = await api.delete(`changes/\${createdId}/`); expect(response.status()).toBe(204); const verify = await api.get(`changes/\${createdId}/`); expect(verify.status()).toBe(404); });</pre> ✓ 4 ...hange - Pruebas de Integración > TC04 - DELETE changes/{id}/ elimina el registro creado (667ms) ✓ DiaperChange - Pruebas de Integración > TC04 - DELETE changes/{id}/ elimina el registro creado chromium 667ms diaperChange.spec.ts:87	

1.14. Sub-Módulo: Feeding (endpoint /feedings/)

1.14.1. TC50 — Validar listado de Feeding (GET)

Campo	Detalle
ID	TC50
Nombre	Validar listado de Feeding
RF asociado	RF-004.3, RF-004.4, RF-006.1
Descripción	Verifica que el endpoint devuelve la lista de registros de Feeding.
Objetivo	Confirmar que GET /feedings/ responde con HTTP 200 y un JSON con la clave "results" como arreglo.

Precondiciones	Cuenta autenticada válida (Token obtenido en /user/settings/). Header Authorization: Token <API_KEY>.
Datos de entrada	Método: GET URL: https://ps-pruebas.onrender.com/api/feedings/ Headers: Authorization: Token <API_KEY>, Accept: application/json
Postcondiciones	La sesión autenticada permanece activa. No se modifican datos.
Prioridad	Media
Estado	pass
Resultado esperado	HTTP 200 OK con JSON que contiene "results" como arreglo.
Resultado Obtenido	HTTP 200 OK con JSON que contiene "results" como arreglo.
Observaciones	No requiere body; solo autenticación válida.
Evidencia	
<pre>test('TC01 - GET feedings/ devuelve lista de alimentaciones', async () => { const response = await api.get('feedings/'); expect(response.status()).toBe(200); const body = await response.json(); expect(Array.isArray(body.results)).toBeTruthy(); });</pre> ✓ 1 ...eeding - Pruebas de Integración > TC01 - GET feedings/ devuelve lista de alimentaciones (190ms) ✓ Feeding - Pruebas de Integración > TC01 - GET feedings/ devuelve lista de alimentaciones chromium 190ms feeding.spec.ts:47	

1.14.2. TC51 — Validar creación de registro de Feeding (POST)

Campo	Detalle
ID	TC51
Nombre	Validar creación de registro de Feeding
RF asociado	RF-004.3, RF-004.4, RF-006.1
Descripción	Verifica que el endpoint crea un nuevo registro de Feeding con datos válidos.
Objetivo	Confirmar que POST /feedings/ responde con HTTP 201 y devuelve el objeto creado.

Precondiciones	Cuenta autenticada válida (Token obtenido en /user/settings/). Header Authorization: Token <API_KEY>. Debe existir al menos un "child" válido (id=1) en el sistema, excepto para el módulo Child.
Datos de entrada	Método: POST URL: https://ps-pruebas.onrender.com/api/feedings/ Headers: Authorization: Token <API_KEY>, Content-Type: application/json Body JSON (child, start, end, type, method, amount, notes): { "child": 1, "start": "2026-07-05T09:00:00-05:00", "end": "2026-07-05T09:15:00-05:00", "type": "breast milk", "method": "left breast", "notes": "QA_TEST"} }
Postcondiciones	Se crea un nuevo registro de Feeding identificable por el prefijo "QA_TEST" para su posterior limpieza.
Prioridad	Alta
Estado	pass
Resultado esperado	HTTP 201 Created con JSON del objeto creado, incluyendo su id.
Resultado Obtenido	HTTP 201 Created con JSON del objeto creado, incluyendo su id.
Observaciones	Guardar el id devuelto; se usará en los TC de PATCH y DELETE de este módulo.
Evidencia	

```
test('TC02 - POST feedings/ crea un nuevo registro de alimentación', async () => {
  const data = {
    child: testChildId,
    start: '2026-07-01T14:00:00-05:00',
    end: '2026-07-01T14:15:00-05:00',
    type: 'breast milk',
    method: 'left breast',
    notes: 'with vitamins',
  };

  const response = await api.post('feedings/', { data });
  expect(response.status()).toBe(201);

  const body = await response.json();
  expect(body.type).toBe(data.type);
  expect(body.method).toBe(data.method);
  expect(body.notes).toBe(data.notes);

  const createdId = body.id;
});
```

✓ 2 ...- Pruebas de Integración > TC02 - POST feedings/ crea un nuevo registro de alimentación (178ms)

✓ Feeding - Pruebas de Integración > TC02 - POST feedings/ crea un nuevo registro de alimentación chromium 178ms

feeding.spec.ts:55

1.14.3. TC52 — Validar actualización parcial de Feeding (PATCH)

Campo	Detalle
ID	TC52
Nombre	Validar actualización parcial de Feeding
RF asociado	RF-004.3, RF-004.4, RF-006.1
Descripción	Verifica que el endpoint actualiza parcialmente un registro existente de Feeding.
Objetivo	Confirmar que PATCH /feedings/{id}/ responde con HTTP 200 y refleja el cambio solicitado.
Precondiciones	Debe existir el registro creado en el TC de POST anterior (mismo id).
Datos de entrada	Método: PATCH URL: https://ps-pruebas.onrender.com/api/feedings/{id}/ Headers: Authorization: Token <API_KEY>, Content-Type: application/json Body JSON: { "notes": "QA_TEST_actualizado" }
Postcondiciones	El registro queda modificado con el nuevo valor.

Prioridad	Media
Estado	pass
Resultado esperado	HTTP 200 OK con JSON que refleja el campo actualizado.
Resultado Obtenido	HTTP 200 OK con JSON que refleja el campo actualizado.
Observaciones	Verificar con un GET adicional que el cambio persiste.
Evidencia	
<pre>test('TC03 - PATCH feedings/{id}/ actualiza type, method y amount', async () => { const response = await api.patch(`feedings/\${createdId}`, { data: { type: 'formula', method: 'bottle', amount: 2.5 }, }); expect(response.status()).toBe(200); const body = await response.json(); expect(body.type).toBe('formula'); expect(body.method).toBe('bottle'); expect(body.amount).toBe(2.5); });</pre>	

1.14.4. TC53 — Validar eliminación de registro de Feeding (DELETE)

Campo	Detalle
ID	TC53
Nombre	Validar eliminación de registro de Feeding
RF asociado	RF-004.3, RF-004.4, RF-006.1
Descripción	Verifica que el endpoint elimina un registro existente de Feeding.
Objetivo	Confirmar que DELETE /feedings/{id}/ responde con HTTP 204 y que el recurso deja de existir.
Precondiciones	Debe existir el registro creado en el TC de POST de este módulo (mismo id).
Datos de entrada	Método: DELETE URL: https://ps-pruebas.onrender.com/api/feedings/{id}/ Headers: Authorization: Token <API_KEY>
Postcondiciones	El registro de prueba queda eliminado; el sistema vuelve al estado previo a la ejecución (limpieza).

Prioridad	Media
Estado	pass
Resultado esperado	HTTP 204 No Content. Un GET posterior al mismo id retorna HTTP 404.
Resultado Obtenido	HTTP 204 No Content. Un GET posterior al mismo id retorna HTTP 404.
Observaciones	Este paso cumple el rol de limpieza (teardown) del dato creado en el POST.
Evidencia	
<pre>test('TC04 - DELETE feedings/{id}/ elimina el registro creado', async () => { const response = await api.delete(`feedings/\${createdId}/`); expect(response.status()).toBe(204); const verify = await api.get(`feedings/\${createdId}/`); expect(verify.status()).toBe(404); });</pre> ✓ 4 ...ding - Pruebas de Integración > TC04 - DELETE feedings/{id}/ elimina el registro creado (724ms) ✓ Feeding - Pruebas de Integración > TC04 - DELETE feedings/{id}/ elimina el registro creado chromium 724ms feeding.spects:88	

1.15. Sub-Módulo: HeadCircumference (endpoint /head-circumference/)

1.15.1. TC54 — Validar listado de HeadCircumference (GET)

Campo	Detalle
ID	TC54
Nombre	Validar listado de HeadCircumference
RF asociado	RF-004.8
Descripción	Verifica que el endpoint devuelve la lista de registros de HeadCircumference.
Objetivo	Confirmar que GET /head-circumference/ responde con HTTP 200 y un JSON con la clave "results" como arreglo.
Precondiciones	Cuenta autenticada válida (Token obtenido en /user/settings/). Header Authorization: Token <API_KEY>.
Datos de entrada	Método: GET URL: https://ps-pruebas.onrender.com/api/head-circumference/ Headers: Authorization: Token <API_KEY>, Accept: application/json

Postcondiciones	La sesión autenticada permanece activa. No se modifican datos.
Prioridad	Media
Estado	pass
Resultado esperado	HTTP 200 OK con JSON que contiene "results" como arreglo.
Resultado Obtenido	HTTP 200 OK con JSON que contiene "results" como arreglo.
Observaciones	No requiere body; solo autenticación válida.
Evidencia	
<pre>test('TC01 - GET head-circumference/ devuelve lista de registros', async () => { const response = await api.get('head-circumference/'); expect(response.status()).toBe(200); const body = await response.json(); expect(Array.isArray(body.results)).toBeTruthy(); });</pre> ✓ 1 ...e - Pruebas de Integración > TC01 - GET head-circumference/ devuelve lista de registros (166ms) ✓ HeadCircumference - Pruebas de Integración > TC01 - GET head-circumference/ devuelve lista de registros chromium 166ms headCircumference.spec.ts:44	

1.15.2. TC55 — Validar creación de registro de HeadCircumference (POST)

Campo	Detalle
ID	TC55
Nombre	Validar creación de registro de HeadCircumference
RF asociado	RF-004.8
Descripción	Verifica que el endpoint crea un nuevo registro de HeadCircumference con datos válidos.
Objetivo	Confirmar que POST /head-circumference/ responde con HTTP 201 y devuelve el objeto creado.
Precondiciones	Cuenta autenticada válida (Token obtenido en /user/settings/). Header Authorization: Token <API_KEY>. Debe existir al menos un "child" válido (id=1) en el sistema, excepto para el módulo Child.
Datos de entrada	Método: POST URL: https://ps-pruebas.onrender.com/api/head-circumference/

	Headers: Authorization: Token <API_KEY>, Content-Type: application/json Body JSON (child, head_circumference, date, notes): {"child": 1, "head_circumference": "40.5", "date": "2026-07-05", "notes": "QA_TEST"}
Postcondiciones	Se crea un nuevo registro de HeadCircumference identificable por el prefijo "QA_TEST" para su posterior limpieza.
Prioridad	Alta
Estado	pass
Resultado esperado	HTTP 201 Created con JSON del objeto creado, incluyendo su id.
Resultado Obtenido	HTTP 201 Created con JSON del objeto creado, incluyendo su id.
Observaciones	Guardar el id devuelto; se usará en los TC de PATCH y DELETE de este módulo.
Evidencia	
<pre>test('TC02 - POST head-circumference/ crea un nuevo registro', async () => { const data = { child: testChildId, head_circumference: '9.5', date: '2026-07-01', }; const response = await api.post('head-circumference/', { data }); expect(response.status()).toBe(201); const body = await response.json(); // se envía como string, el servidor lo devuelve como number expect(body.head_circumference).toBe(9.5); const createdId = body.id; });</pre> ✓ 2 ...rence - Pruebas de Integración > TC02 - POST head-circumference/ crea un nuevo registro (182ms) ✓ HeadCircumference - Pruebas de Integración > TC02 - POST head-circumference/ crea un nuevo registro chromium 182ms headCircumference.spects:52	

1.15.3. TC56 — Validar actualización parcial de HeadCircumference (PATCH)

Campo	Detalle
ID	TC56

Nombre	Validar actualización parcial de HeadCircumference
RF asociado	RF-004.8
Descripción	Verifica que el endpoint actualiza parcialmente un registro existente de HeadCircumference.
Objetivo	Confirmar que PATCH /head-circumference/{id}/ responde con HTTP 200 y refleja el cambio solicitado.
Precondiciones	Debe existir el registro creado en el TC de POST anterior (mismo id).
Datos de entrada	Método: PATCH URL: https://ps-pruebas.onrender.com/api/head-circumference/{id}/ Headers: Authorization: Token <API_KEY>, Content-Type: application/json Body JSON: { "head_circumference": "41.0" }
Postcondiciones	El registro queda modificado con el nuevo valor.
Prioridad	Media
Estado	pass
Resultado esperado	HTTP 200 OK con JSON que refleja el campo actualizado.
Resultado Obtenido	HTTP 200 OK con JSON que refleja el campo actualizado.
Observaciones	Verificar con un GET adicional que el cambio persiste.
Evidencia	
<pre>test('TC03 - PATCH head-circumference/{id}/ actualiza la medida', async () => { const response = await api.patch(`head-circumference/\${createdId}/`, { data: { head_circumference: 23 }, }); expect(response.status()).toBe(200); const body = await response.json(); expect(body.head_circumference).toBe(23); });</pre> ✓ 3 ...ce - Pruebas de Integración > TC03 - PATCH head-circumference/{id}/ actualiza la medida (194ms) ✓ HeadCircumference - Pruebas de Integración > TC03 - PATCH head-circumference/{id}/ actualiza la medida chromium 194ms headCircumference.spec.ts:69	

1.15.4. TC57 — Validar eliminación de registro de HeadCircumference (DELETE)

Campo	Detalle
-------	---------

ID	TC57
Nombre	Validar eliminación de registro de HeadCircumference
RF asociado	RF-004.8
Descripción	Verifica que el endpoint elimina un registro existente de HeadCircumference.
Objetivo	Confirmar que DELETE /head-circumference/{id}/ responde con HTTP 204 y que el recurso deja de existir.
Precondiciones	Debe existir el registro creado en el TC de POST de este módulo (mismo id).
Datos de entrada	Método: DELETE URL: https://ps-pruebas.onrender.com/api/head-circumference/{id}/ Headers: Authorization: Token <API_KEY>
Postcondiciones	El registro de prueba queda eliminado; el sistema vuelve al estado previo a la ejecución (limpieza).
Prioridad	Media
Estado	pass
Resultado esperado	HTTP 204 No Content. Un GET posterior al mismo id retorna HTTP 404.
Resultado Obtenido	HTTP 204 No Content. Un GET posterior al mismo id retorna HTTP 404.
Observaciones	Este paso cumple el rol de limpieza (teardown) del dato creado en el POST.
Evidencia	
<pre>test('TC04 - DELETE head-circumference/{id}/ elimina el registro creado', async () => { const response = await api.delete(`head-circumference/\${createdId}/`); expect(response.status()).toBe(204); const verify = await api.get(`head-circumference/\${createdId}/`); expect(verify.status()).toBe(404); });</pre> ✓ 4 ...ebas de Integración > TC04 - DELETE head-circumference/{id}/ elimina el registro creado (665ms) ✓ HeadCircumference - Pruebas de Integración > TC04 - DELETE head-circumference/{id}/ elimina el registro creado 665ms chromium headCircumference.spec.ts:79	

1.16. Sub-Módulo: Height (endpoint /height/)

1.16.1. TC58 — Validar listado de Height (GET)

Campo	Detalle
-------	---------

ID	TC58
Nombre	Validar listado de Height
RF asociado	RF-004.8
Descripción	Verifica que el endpoint devuelve la lista de registros de Height.
Objetivo	Confirmar que GET /height/ responde con HTTP 200 y un JSON con la clave "results" como arreglo.
Precondiciones	Cuenta autenticada válida (Token obtenido en /user/settings/). Header Authorization: Token <API_KEY>.
Datos de entrada	Método: GET URL: https://ps-pruebas.onrender.com/api/height/ Headers: Authorization: Token <API_KEY>, Accept: application/json
Postcondiciones	La sesión autenticada permanece activa. No se modifican datos.
Prioridad	Media
Estado	pass
Resultado esperado	HTTP 200 OK con JSON que contiene "results" como arreglo.
Resultado Obtenido	HTTP 200 OK con JSON que contiene "results" como arreglo.
Observaciones	No requiere body; solo autenticación válida.
Evidencia	
 <pre>test('TC01 - GET height/ devuelve lista de registros', async () => { const response = await api.get('height/'); expect(response.status()).toBe(200); const body = await response.json(); expect(Array.isArray(body.results)).toBeTruthy(); });</pre> ✓ 1 ...44:9 > Height - Pruebas de Integración > TC01 - GET height/ devuelve lista de registros (180ms) ✓ Height - Pruebas de Integración > TC01 - GET height/ devuelve lista de registros chromium 180ms height.spec.ts:44	

1.16.2. TC59 — Validar creación de registro de Height (POST)

Campo	Detalle
--------------	----------------

ID	TC59
Nombre	Validar creación de registro de Height
RF asociado	RF-004.8
Descripción	Verifica que el endpoint crea un nuevo registro de Height con datos válidos.
Objetivo	Confirmar que POST /height/ responde con HTTP 201 y devuelve el objeto creado.
Precondiciones	Cuenta autenticada válida (Token obtenido en /user/settings/). Header Authorization: Token <API_KEY>. Debe existir al menos un "child" válido (id=1) en el sistema, excepto para el módulo Child.
Datos de entrada	Método: POST URL: https://ps-pruebas.onrender.com/api/height/ Headers: Authorization: Token <API_KEY>, Content-Type: application/json Body JSON (child, height, date, notes): { "child": 1, "height": "55.0", "date": "2026-07-05", "notes": "QA_TEST" }
Postcondiciones	Se crea un nuevo registro de Height identificable por el prefijo "QA_TEST" para su posterior limpieza.
Prioridad	Alta
Estado	pass
Resultado esperado	HTTP 201 Created con JSON del objeto creado, incluyendo su id.
Resultado Obtenido	HTTP 201 Created con JSON del objeto creado, incluyendo su id.
Observaciones	Guardar el id devuelto; se usará en los TC de PATCH y DELETE de este módulo.
Evidencia	

```
test('TC02 - POST height/ crea un nuevo registro', async () => {
  const data = {
    child: testChildId,
    height: '12.5',
    date: '2026-07-01',
  };

  const response = await api.post('height/', { data });
  expect(response.status()).toBe(201);

  const body = await response.json();
  // se envía como string, el servidor lo devuelve como number
  expect(body.height).toBe(12.5);

  const createdId = body.id;
});
```

✓ 2 ...ts:52:9 > Height - Pruebas de Integración > TC02 - POST height/ crea un nuevo registro (215ms)

✓ Height - Pruebas de Integración > TC02 - POST height/ crea un nuevo registro [chromium](#) 215ms

height.spec.ts:52

1.16.3. TC60 — Validar actualización parcial de Height (PATCH)

Campo	Detalle
ID	TC60
Nombre	Validar actualización parcial de Height
RF asociado	RF-004.8
Descripción	Verifica que el endpoint actualiza parcialmente un registro existente de Height.
Objetivo	Confirmar que PATCH /height/{id}/ responde con HTTP 200 y refleja el cambio solicitado.
Precondiciones	Debe existir el registro creado en el TC de POST anterior (mismo id).
Datos de entrada	Método: PATCH URL: https://ps-pruebas.onrender.com/api/height/{id}/ Headers: Authorization: Token <API_KEY>, Content-Type: application/json Body JSON: {"height": "56.5"}
Postcondiciones	El registro queda modificado con el nuevo valor.

Prioridad	Media
Estado	pass
Resultado esperado	HTTP 200 OK con JSON que refleja el campo actualizado.
Resultado Obtenido	HTTP 200 OK con JSON que refleja el campo actualizado.
Observaciones	Verificar con un GET adicional que el cambio persiste.
Evidencia	
<pre>test('TC03 - PATCH height/{id}/ actualiza la altura', async () => { const response = await api.patch(`height/\${createdId}/`, { data: { height: 23.5 }, }); expect(response.status()).toBe(200); const body = await response.json(); expect(body.height).toBe(23.5); });</pre> ✓ 3 ...:69:9 > Height - Pruebas de Integración > TC03 - PATCH height/{id}/ actualiza la altura (191ms) ✓ Height - Pruebas de Integración > TC03 - PATCH height/{id}/ actualiza la altura chromium 191ms height.spec.ts:69	

1.16.4. TC61 — Validar eliminación de registro de Height (DELETE)

Campo	Detalle
ID	TC61
Nombre	Validar eliminación de registro de Height
RF asociado	RF-004.8
Descripción	Verifica que el endpoint elimina un registro existente de Height.
Objetivo	Confirmar que DELETE /height/{id}/ responde con HTTP 204 y que el recurso deja de existir.
Precondiciones	Debe existir el registro creado en el TC de POST de este módulo (mismo id).
Datos de entrada	Método: DELETE URL: https://ps-pruebas.onrender.com/api/height/{id}/ Headers: Authorization: Token <API_KEY>
Postcondiciones	El registro de prueba queda eliminado; el sistema vuelve al estado previo a la ejecución (limpieza).

Prioridad	Media
Estado	pass
Resultado esperado	HTTP 204 No Content. Un GET posterior al mismo id retorna HTTP 404.
Resultado Obtenido	HTTP 204 No Content. Un GET posterior al mismo id retorna HTTP 404.
Observaciones	Este paso cumple el rol de limpieza (teardown) del dato creado en el POST.
Evidencia	
<pre>test('TC04 - DELETE height/{id}/ elimina el registro creado', async () => { const response = await api.delete(`height/\${createdId}/`); expect(response.status()).toBe(204); const verify = await api.get(`height/\${createdId}/`); expect(verify.status()).toBe(404); });</pre> ✓ 4 ...Height - Pruebas de Integración › TC04 - DELETE height/{id}/ elimina el registro creado (378ms) ✓ Height - Pruebas de Integración › TC04 - DELETE height/{id}/ elimina el registro creado chromium 378ms height.spec.ts:79	

2. Resumen de Resultados

2.1. Resultados aplicados

Se diseñaron y ejecutaron un total de 61 casos de prueba sobre 16 submódulos de la API REST de Baby Buddy, distribuidos para ser ejecutados y probados por las herramientas anteriormente establecidas. Del total de casos (100%) finalizaron en estado "Pass", cubriendo las operaciones de los metodos GET, POST, PATCH y DELETE sobre los siguientes submódulos: BMI, Child, Pumping, DiaperChange, Feeding, HeadCircumference, Height, Medication, Note, Sleep, Tags, Temperature, Timer, TummyTime, Weight y Profile.

Submódulo	Casos ejecutados	Casos exitosos	Casos fallidos	Porcentaje de éxito
Medication	4	4	0	100%
Note	4	4	0	100%

Submódulo	Casos ejecutados	Casos exitosos	Casos fallidos	Porcentaje de éxito
Profile	1	1	0	100%
Sleep	3	3	0	100%
Tags	5	5	0	100%
Temperature	4	4	0	100%
Timer	5	5	0	100%
TummyTime	3	3	0	100%
Weight	4	4	0	100%
BMI	4	4	0	100%
Child	4	4	0	100%
Pumping	4	4	0	100%
DiaperChange	4	4	0	100%
Feeding	4	4	0	100%
HeadCircumference	4	4	0	100%
Height	4	4	0	100%
Total	61	61	0	100%

3. Conclusiones

La ejecución de las 61 pruebas de integración permite concluir que la arquitectura REST de Baby Buddy es funcionalmente estable y consistente en el manejo de operaciones CRUD, con una tasa de éxito del 100% en los casos diseñados. La adopción de Playwright como herramienta de automatización, en reemplazo de la

ejecución manual con Postman utilizada en auditorías previas del equipo, permitió estructurar las pruebas como repetibles y versionables, facilitando la trazabilidad de resultados y agilizando la generación de evidencia a través del reporte HTML nativo de Playwright.